

國立中央大學

資訊工程學系
碩士論文

小提琴演奏追蹤系統：應用音源分離結果實現即時音樂追蹤與伴奏

A Violin Performance Tracking System: Utilizing
Music Source Separation Results for Real-Time Music
Tracking and Accompaniment

研究生：林妤潔

指導教授：蘇木春 博士

中華民國一百一十三年六月

國立中央大學碩士班研究生 論文指導教授推薦書

資訊工程學系碩士班 學系/研究所 林妤潔 研究生

所提之論文 小提琴演奏追蹤系統：應用音源分離結果實現即時
音樂追蹤與伴奏

係由本人指導撰述，同意提付審查。

指導教授 蔡木君 (簽章)

113 年 6 月 17 日

1130531

國立中央大學碩士班研究生 論文口試委員審定書

資訊工程學系碩士班 學系/研究所 林妤潔 研究生

所提之論文 小提琴演奏追蹤系統：應用音源分離結果實現即時

音樂追蹤與伴奏

經由委員會審議，認定符合碩士資格標準。

學位考試委員會召集人

傅立成

委

員

林道通

林道通教授 (Jul 4, 2024 21:43 GMT+8)

陳木君

中 華 民 國

113 年 7 月 3 日

1130620

小提琴演奏追蹤系統：應用音源分離結果實現即時音樂追蹤與伴奏

摘要

小提琴一直以來都是許多人學習與演奏的樂器，有許多膾炙人口的曲子與優秀的小提琴音樂家。在眾多曲子中，小提琴與其他樂器的合奏曲子佔多數，因此需要其他樂器的演奏者一起合奏才能完整呈現曲子的風貌。然而，由於時間或成本的因素，尋找長期合作的合奏者(伴奏者)並不是那麼容易，網路上的公開資源又多為混合音訊，合奏的效果不佳。因此本研究針對最常見的小提琴與鋼琴的合奏方式來開發一套系統，此系統可將混合音源中的小提琴與鋼琴音源分離，並使用分離音源追蹤現場小提琴演奏，輸出鋼琴伴奏。

本研究旨在開發一套使用音源分離結果實現小提琴演奏追蹤的即時音樂追蹤系統，我們設計了音源分離模組與音樂追蹤模組，在音源分離模組方面，我們自行蒐集並建立一套新的公開整合資料集，用於訓練 Band-Split RNN 模型，並改進了模型的頻帶切割方法。在模型的評估上，我們使用訊號失真比來計算模型的分離效果，結果顯示模型在資料缺乏與資料充足的情況下皆優於現有的基線模型，並證明頻帶切割方法的有效性。在音樂追蹤模組方面，我們改進了線上動態時間規整演算法與貪心向後對齊方法，重現了即時音樂追蹤模組的設計，並改良部分元件。在實際的測試中，即時音樂追蹤系統展現了低延遲與精準追蹤的表現，並在不同特徵的追蹤表現上保持了與離線追蹤相同穩定的追蹤效果。

關鍵字：音樂資訊檢索, 音源分離, 音樂追蹤, 自動伴奏, 深度學習

A Violin Performance Tracking System: Utilizing Music Source Separation Results for Real-Time Music Tracking and Accompaniment

Abstract

The violin has long been a popular instrument for learning and performance, with many well-known pieces and distinguished violinists. Among these pieces, ensemble compositions involving the violin and other instruments are predominant, requiring collaboration with other instrumentalists to fully present the musical piece. However, due to time or cost constraints, finding long-term ensemble partners (accompanists) can be challenging due to time or cost constraints. Online public resources often provide mixed audio, which does not yield good ensemble effects. Therefore, this research focuses on developing a system for the common violin and piano ensemble. This system can separate the violin and piano sources from a mixed audio source, track the violin's performance using the separated audio, and output the piano accompaniment.

The goal of this research is to develop a real-time music tracking system that utilizes source separation results to track violin performances. We designed a source separation module and a music tracking module. For the source separation module, we collected and established a new open integrated dataset to train the Band-Split RNN model, improving the model's band-split method. We evaluated the model using the Signal-to-Distortion Ratio to measure the separa-

tion performance. The results show that the model outperforms existing baseline models in both data-limit and data-rich cases, demonstrating the effectiveness of the band-split method. For the music tracking module, we improved the On-line Dynamic Time Warping algorithm and the Greedy Backward Alignment method, reimplementing the design of the real-time music tracking module and enhancing some blocks. In practical tests, the real-time music tracking system exhibited low latency and accurate tracking performance, maintaining stable tracking results comparable to offline tracking across different feature tracking performances.

Keywords: Music Information Retrieval, Music Source Separation, Music Tracking, Automatic Accompaniment, Deep Learning

誌謝

誌謝...

目錄

	頁次
摘要	iii
Abstract	v
誌謝	vii
目錄	viii
一、 緒論	1
1.1 研究動機	1
1.2 研究目的	4
1.3 論文架構	5
二、 背景知識以及文獻回顧	6
2.1 背景知識	6
2.1.1 小提琴與鋼琴的演奏特性	6
2.1.2 小提琴與鋼琴的音色分析	7
2.2 文獻回顧	10
2.2.1 音源分離相關研究	10
2.2.2 音樂追蹤相關研究	12
三、 研究方法	14
3.1 系統架構	14

3.2	音源分離模組	16
3.2.1	Band-Split RNN	16
3.2.2	頻帶切割估計方法	18
3.3	音樂追蹤模組	20
3.3.1	動態時間規整 (Dynamic Time Warping, DTW)	20
3.3.2	線上動態時間規整 (Online Dynamic Time Warping, ODTW)	23
3.3.3	貪心向後對齊 (Greedy Backward Alignment, GBA)	29
3.3.4	資料管理者元件 (Data Manager Block).....	30
3.3.5	音樂偵測器元件 (Music Detector Block)	33
3.3.6	粗略位置估計器元件 (Rough Position Estimator Block, RPE).....	34
3.3.7	決策決定者元件 (Decision Maker Block)	37
四、	實驗設計與結果	40
4.1	音源分離評估	40
4.1.1	音源分離資料集	40
4.1.2	模型訓練細節	42
4.1.3	音源分離評估指標	43
4.1.4	音源分離結果比較	44
4.1.5	頻帶切割對於分離結果的影響	47
4.2	音樂追蹤評估	49
4.2.1	音樂追蹤模組評估方法	49
4.2.2	音樂追蹤模組評估結果	51
4.3	整體系統評估	57
4.3.1	系統評估方法	57
4.3.2	系統評估結果	58

五、	總結	65
5.1	結論	65
5.2	未來展望	66
參考文獻		67

圖目錄

頁次

2.1	小提琴與鋼琴演奏 G4 音高三秒的特徵圖：(a) 鋼琴，(b) 小提琴，(c) 混合	8
2.2	同一人演奏兩次 Beethoven Spring Sonata No.1 之特徵圖 . . .	9
2.3	不同人演奏 Bach BWV1006 之特徵圖	9
3.1	系統架構圖	14
3.2	BSRNN 輸入頻譜圖	16
3.3	BSRNN 架構圖，擷取自 [35]	17
3.4	兩種資料的波形圖 (上) 與經過 FFT 轉換後的 F_{band} 圖 (下) .	18
3.5	兩個相似的時間序列：藍色: X 紅色: Y	20
3.6	DTW 單格成本距離矩陣示意圖	21
3.7	DTW 累積距離成本矩陣計算示意圖	22
3.8	DTW 累積距離成本矩陣回溯路徑示意圖	23
3.9	ODTW 對齊路徑計算，擷取自 [50]	24
3.10	每次迭帶所計算到的距離成本矩陣格數位置比較	25
3.11	傳統 ODTW 計算輸出對齊點範圍示意圖	26
3.12	改動後 ODTW 計算輸出對齊點範圍示意圖	27
3.13	傳統 ODTW 決定計算方向示意圖, $c = 3$	28
3.14	改動後 ODTW 決定計算方向示意圖, $c = 3$	29
3.15	GBA 計算 W_{cost} 示意圖	30

3.16	現場與參考的高低解析度特徵圖：(a) 現場高解析度，(b) 現場低解析度，(c) 參考高解析度，(d) 參考低解析度	32
3.17	粗略位置估計器的距離成本矩陣熱力圖	35
3.18	粗略位置估計器的輸出位置	36
3.19	追蹤結束後決策決定者輸出的熱力圖與相關元件的輸出點 .	38
3.20	DTW 與系統的輸出路徑比較圖	39
4.1	測試資料集 TheKitchenettes_Alive_p0_v0 結果比較圖：(a) 混合音訊、(b) 鋼琴音訊、(c) 鋼琴 Ground truth、(d) 小提琴音訊、(e) 小提琴 Ground truth	46
4.2	貝多芬第五號小提琴奏鳴曲第一樂章結果圖：(a) 混合音訊、(b) 鋼琴音訊、(c) 小提琴音訊	46
4.3	韋瓦第四季-夏 第三樂章結果圖：(a) 混合音訊、(b) 鋼琴音訊、(c) 小提琴音訊	47
4.4	四種不同速度範圍的 MIDI 樂譜前五小節	50
4.5	慢 (90-120bpm) 平均延遲：55.04 毫秒	52
4.6	一般 (115-145bpm) 平均延遲：35.81 毫秒	52
4.7	快 (135-175bpm) 平均延遲：62.96 毫秒	53
4.8	加速 (80→160bpm) 平均延遲：58.73 毫秒	53
4.9	慢 (90-120bpm)	54
4.10	一般 (115-145bpm)	55
4.11	快 (135-175bpm)	55
4.12	加速 (80→160bpm)	56
4.13	硬體設備	57
4.14	110bpm 平均延遲：26.62 幀	58
4.15	120bpm 平均延遲：27.98 幀	59
4.16	130bpm 平均延遲：37.49 幀	60
4.17	140bpm 平均延遲：40.68 幀	61

4.18 自由速度平均延遲：28.43 幀	62
4.19 自由速度 (參考音訊為同一人所演奏) 平均延遲：22.54 幀 . .	64

表目錄

	頁次
3.1 頻帶切割估計方法參數設定	19
3.2 ODTW 計算輸出範圍參數設定	26
3.3 STFT 參數設定	31
3.4 高解析度特徵參數設定	31
3.5 低解析度特徵參數設定	31
4.1 整合的訓練資料集格式	41
4.2 每種樂器的訓練集與驗證集大小	41
4.3 測試資料集	42
4.4 每次迭帶的訓練與驗證資料筆數	42
4.5 資料讀取與特徵擷取參數	42
4.6 模型訓練參數	43
4.7 Data-limit 結果數值比較	44
4.8 Data-rich 結果數值比較	44
4.9 Data-limit 不同的 Band-Split Bandwidth 結果比較	48

一、緒論

1.1 研究動機

根據大學術科考試委員會 107 年至 112 年的音樂術科考試人數資料統計 [1]，樂器主修報考最多的項目分別為弦樂與鋼琴，其中弦樂主修又以小提琴佔比最高。由此可知，小提琴是許多人學習和演奏的樂器。在眾多涉及小提琴的樂曲中，除了無伴奏小提琴曲 (例如巴赫無伴奏小提琴奏鳴曲等) 之外，幾乎都需要與其他樂器合奏，而鋼琴則是最為常見的合奏樂器。例如小提琴奏鳴曲就是由小提琴與鋼琴共同演奏的曲子，此外也有許多曲目從原始樂器編制改編為鋼琴與小提琴的合奏版本。

通常演奏合奏曲目，必須自行尋找或是聘請其他演奏者共同演奏，但聘請其他演奏者的費用並不便宜，通常一節伴奏 (約 50 分鐘到 1 小時) 會根據專業度的不同來收費，平均收費約為台幣 600 元至 1800 元不等 [2]。因此若找不到其他演奏者，通常只能選擇單獨演奏自己的部分或是在網路上搜尋音訊使用。有些演奏者會將音訊上傳至公開平台供大家使用，但這些音訊通常已經是混合音訊，無法根據個人習慣或練習的速度演奏，也會被音訊中演奏同一部份的聲音干擾。

因此若能將網路上公開的混合音訊分離出每種樂器的單音軌音訊，並使用合奏樂器的單音軌音訊跟隨自己的演奏速度播放，便可以在無法找到其他演奏者時自主練習，同時也省下了人事成本。

近年來，隨著音樂數位化與深度學習的進步，音樂資訊檢索 (Music Information Retrieval, MIR) 這門領域的發展越來越受到許多人的關注，

這門領域包含音樂來源分離 (Music Source Separation, MSS) [3]–[5]、自動伴奏 (Automatic Accompaniment) [6]–[8]、樂譜追蹤 (Music Score Tracking) [9], [10]、音樂生成 (Music Generation) [11], [12]、樂器辨識 (Musical Instrument Recognition) [13]–[15] 等子領域，這些研究領域也被廣泛的應用於商業化的產品上，例如音樂推薦系統 [16]、音樂創作工具 [17]–[19]、音樂教育應用軟體 [20], [21] 等。

國際音樂資訊檢索協會 (International Society for Music Information Retrieval, ISMIR) [22] 自 2000 年開始每年舉行 MIR 研討會，促進相關領域的交流。其中音樂來源分離與自動伴奏更是近幾年許多人關注的領域，音樂來源分離的主要目標為分離混和音訊中的各個音源 (樂器)，分離出來的音源可應用於音樂的重新混音，如卡拉 OK、DJ 混音等，或作為其他問題的前處理工具 [23]。Sony 與 ISMIR 在 2021 年舉辦了音樂解混 (Music Demixing, MDX) 競賽 [24]，大力推動了這項技術的發展，近期也延續了先前的成果舉辦更大的聲音分離 (Sound Demixing, SDX) 競賽 [25]。自動伴奏的主要目標為根據特定的旋律生成伴奏 [26], [27]，或是追蹤特定的旋律跟隨樂譜 [28]。這項技術已經在音樂教育與音樂創作 [29] 等商業產品使用。

在上述的兩個領域中，音樂來源分離的研究重點主要集中在流行樂音源的分離，自動伴奏大部分也是針對可通過音樂數位介面 (Musical Instrument Digital Interface, MIDI) 協定傳輸的樂器 (例如：電子琴、電子鼓等等) 來研究。然而古典樂器因為音色的複雜性與資料的稀缺性，在音源分離與自動伴奏領域的研究相對較少，目前也尚未有將音源分離的結果結合於自動伴奏的研究，因此本研究旨在開發一套專注於追蹤小提琴演奏的即時音樂追蹤系統，此系統應用音源分離技術將小提琴與鋼琴的混合音源分離為小提琴分離音源與鋼琴分離音源，作為即時音樂追蹤系統的參考音訊，當使用者現場演奏小提琴時，即時音樂追蹤系統透過現場演奏與小提琴分離音源追蹤現場演奏的位置，並取得對應的鋼琴分離音源位置作為估計伴奏音訊輸出，讓個人練習時有方便的系統可以使用，

也提供音樂創作者更多不同的創作方式。本研究將現場小提琴演奏稱為現場音訊，音源分離結果稱為參考音訊，其中小提琴分離音源稱為小提琴參考音訊，鋼琴分離音源稱為鋼琴參考音訊；即時音樂追蹤系統輸出稱為估計伴奏音訊。

1.2 研究目的

本研究的目的是開發一套專注於追蹤小提琴演奏的即時音樂追蹤系統，此系統分兩部分研究，第一部分為音源分離技術的研究，此研究專注於探討如何分離小提琴與鋼琴的混合音訊，作為後續音樂追蹤的參考音訊與伴奏音訊使用。研究詳細內容包含探討如何根據不同樂器的特性調整資料前處理的方式，以提升音源分離模型的效果、應用深度學習網路訓練音源分離模型並與過往表現突出的模型比較效果。第二部分為音樂追蹤技術的研究，此研究專注於探討如何使用音源分離的結果來追蹤現場音訊的位置，並輸出對應的伴奏音訊達到即時合奏的效果。研究詳細內容包含探討系統的設計，平均分配系統計算資源來達到即時的效果、設計參考音訊、伴奏音訊與現場音訊的特徵提取方式、設計音樂偵測元件 (Music Detector) 判斷現場演奏是否開始、改良粗略估計位置元件 (Rough Position Estimator, RPE)、線上動態時間規整演算法 (Online Dynamic Time Warping, ODTW) 與貪心向後對齊方法 (Greedy Backward Alignment, GBA) 提升追蹤位置的準確率、設計決策決定者元件 (Decision Maker) 決定最後輸出位置並輸出對應的伴奏音訊。

1.3 論文架構

本論文分為五個章節，其架構如下：

第一章、緒論，敘述本論文之研究目的、動機以及架構。

第二章、背景知識以及文獻回顧，介紹本研究所需的背景知識，包含小提琴與鋼琴的基本演奏方式與特性分析，以及小提琴與鋼琴的音色比較，並探討目前在音源分離領域與音樂追蹤領域的研究現況。

第三章、研究方法，說明本研究細節，如整體的系統架構、音源分離模組設計、音樂追蹤模組設計與改良細節。

第四章、實驗設計與結果，說明實驗使用的資料集、實驗設計內容以及評估方法，並對於實驗結果進行探討。

第五章、總結，對於整體研究結果進行總結，提出尚可改進的部分並討論本研究的未來展望。

二、 背景知識以及文獻回顧

2.1 背景知識

本研究專注於開發一套適用於小提琴與鋼琴的系統，因此本節將介紹小提琴與鋼琴的基本特性與特色。

2.1.1 小提琴與鋼琴的演奏特性

小提琴是一種弓弦樂器，演奏方式透過左手與右手的協調配合來實現。左手透過指腹按壓琴弦來調整音高，常見的左手指法技巧有滑音、抖音等等。滑音為通過指腹沿琴弦滑動到不同位置，產生音符連續變化的音色，抖音則是通過手腕的擺動，使按壓點的觸碰面積改變產生微小的音高波動，使音符聽起來有抖動的音色；右手握弓並將弓毛貼住琴弦，透過手臂與手腕的上下來回動作使弓與琴弦摩擦並發出聲音，常見的弓法技巧有連弓、跳弓等等。連弓技巧是在一弓之內演奏多個音符，產生連貫流暢的音色。跳弓則是使弓在琴弦上跳動，產生彈性與清晰的音色。

鋼琴是帶有琴弦的鍵盤樂器，演奏方式透過雙手與雙腳的協調配合來實現。雙手透過指腹敲擊琴鍵，使琴槌敲擊琴弦發出聲音，其中可以透過不同的敲擊力度與速度產生不同的音量與音色。雙腳透過踩踏鋼琴下的踏板，改變聲音的音色。踏板大致可分為延音踏板與弱音踏板，延音踏板可使音符響起的時間更長，而弱音踏板則是使彈奏的音符更為柔和。

由於樂器本身結構的不同，使兩者演奏的音色也大不相同，小提琴在演奏時，左手可以隨意控制音高，右手可以控制音量，且隨著演奏者的技巧能力，能展現非常多種且細膩的音色變化；鋼琴則是已經有固定的琴鍵控制音高，在音色方面可以使用不同力道敲擊琴鍵並配合踏板使用來表現，但與小提琴相比，音色變化的多樣性較低，但由於鋼琴為雙手彈奏的樂器，在多聲部或複雜和聲的表現上比起小提琴更能勝任這類的作品。

2.1.2 小提琴與鋼琴的音色分析

當兩種樂器以同樣的音量演奏同一段旋律，我們可以輕易地分辨哪一段旋律來自哪種樂器，甚至是當不同人以同樣的樂器、音量、旋律演奏，我們還是可以分辨出兩段旋律的差異。以物理學來看，聲音(聲波)是由頻率與振幅組成，頻率決定了音高，振幅決定了聲音的大小，而我們所聽到的聲音大部分是由許多不同頻率疊加在一起的，因此頻率的組成又可以拆成基頻與泛音，基頻決定了音高，泛音則是決定這個聲音的音色，也就是人類可以區分不同聲音的原因。

舉個例子，圖 2.1 為小提琴與鋼琴演奏同一個音高 G4($\sim 392\text{Hz}$) 三秒鐘的特徵圖，上中下分別為鋼琴、小提琴與鋼琴小提琴合併後的特徵圖，在上圖與中間圖最亮的地方皆為頻率軸(y 軸)上約 392Hz 的地方，也就是代表 G4 音高的基頻，而泛音通常會出現在基頻的整數倍，因此在頻率軸 $392 \times 2, 392 \times 3 \dots \text{Hz}$ 的地方也有特徵顯示。可以看到兩種樂器在泛音頻率的成份並不相同，因此造就了不同的音色。

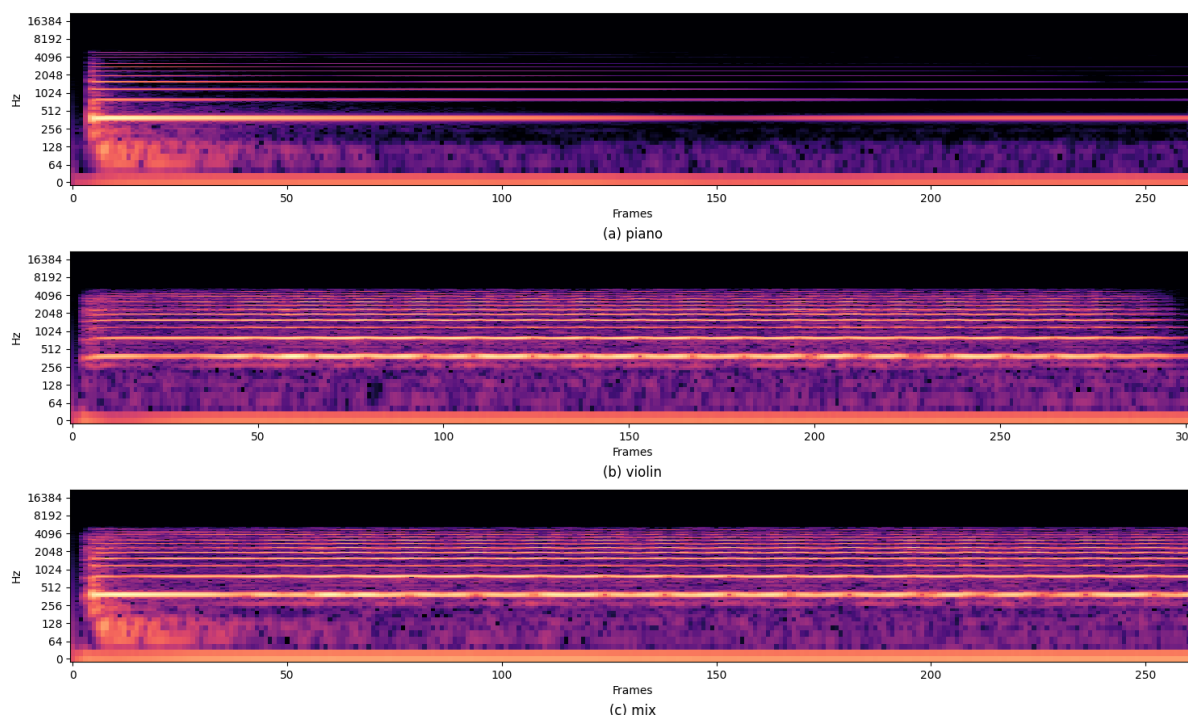


圖 2.1: 小提琴與鋼琴演奏 G4 音高三秒的特徵圖：(a) 鋼琴，(b) 小提琴，(c) 混合

雖然人耳可以輕易分辨出不同的音色，但是當兩種聲音混合在一起時，要精準的將兩種樂器的聲音分開是一件困難的任務，圖 2.1 的下方圖顯示了音訊混合後的特徵長相，若要將兩種樂器的音源分離，我們必須精準地知道每個頻率原本的成分才有可能乾淨的分離。音源分離的難度與樂器音色的複雜度、樂器種類的相似度而提高，因此至今音源分離的任務還是非常具有挑戰性。

另外在音樂追蹤方面，使用兩段相似的音訊特徵來追蹤是常見的方法，特徵越相近越容易有好的追蹤結果，但若是屬於音色變化較大的樂器 (例如小提琴)，儘管是同一人演奏相同曲目兩次，特徵也不一定相同，圖 2.2 顯示同一人使用相同速度、曲目演奏兩次的特徵圖，雖然大致上看起來都相同，但細看的話在時間軸為 100 frames 時，兩邊的特徵還是有些微的不同，這些些微的不同之處都有可能影響到追蹤結果。圖 2.3 顯示不同人演奏同一首曲子的特徵圖，可以看到這兩段音訊明顯在環境的收音上可能差很多，且在曲子的詮釋上也不完全相同，使用這兩段音訊來追蹤，難度會比同一人演奏的情況要高很多，因此使用不同音訊特徵

來追蹤是具有一定的挑戰性。

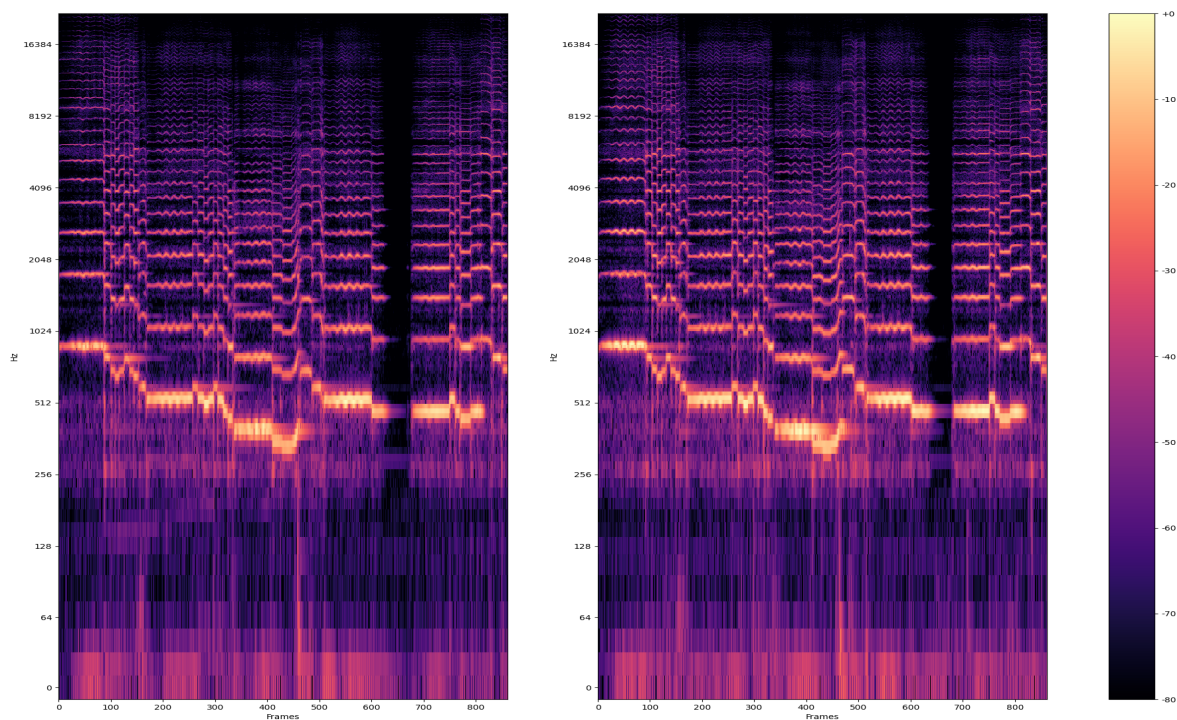


圖 2.2: 同一人演奏兩次 Beethoven Spring Sonata No.1 之特徵圖

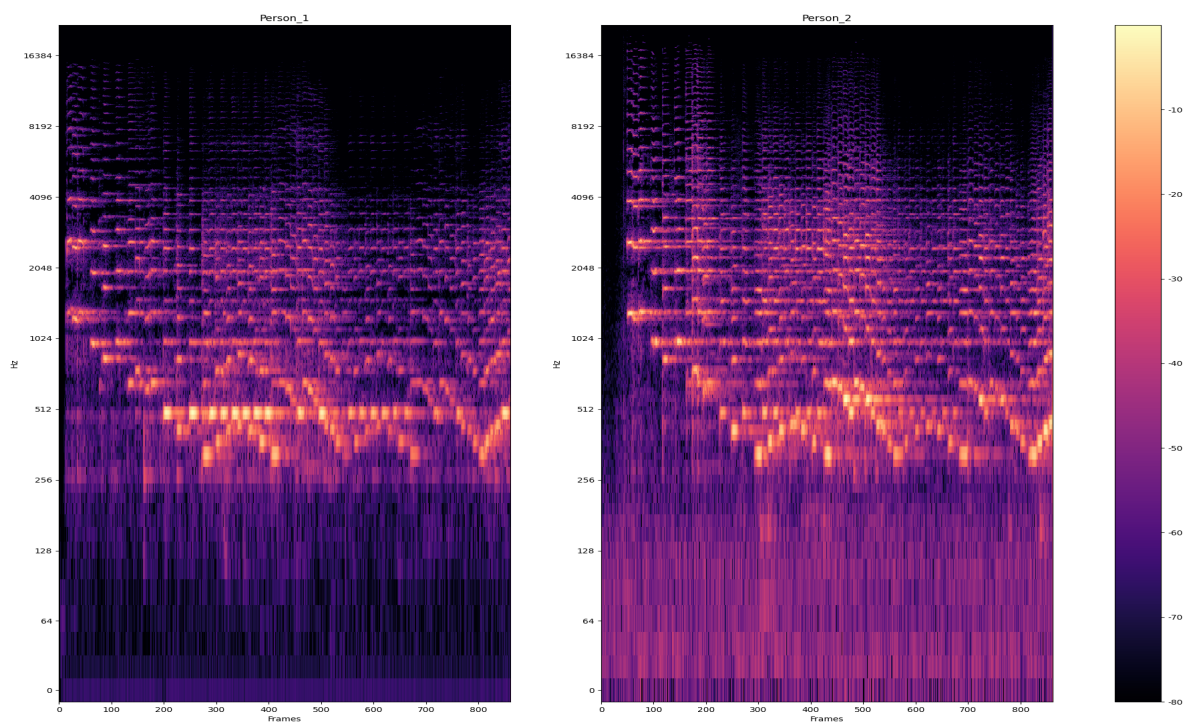


圖 2.3: 不同人演奏 Bach BWV1006 之特徵圖

2.2 文獻回顧

2.2.1 音源分離相關研究

音源分離 (Music Source Separation, MSS) 為 MIR 中的一個重要領域，日常生活中我們所聽到的音樂大多數是來自多個音源的混合音訊，許多人可能希望調整混合音訊中的某種樂器的聲音平衡、改變聲音的空間位置，甚至將混合音訊與每個存在於其中的音樂來源作為其他 MIR 任務的訓練資料，因此，若是可以取得每個音樂來源的單獨音訊，上述所說的應用都能夠實現。

而音源分離是盲源分離 (Blind Source Separation, BSS) 的一個子任務，更早出現在 MIR 的領域中，盲源分離不只用於處理音訊資料，在生物醫學訊號與通信訊號的處理上也會使用到此技術。主要方法有兩種，獨立成分分析 (Independent Components Analysis, ICA) [30]，此方法的目標是找到一個分離矩陣，使混合訊號透過分離矩陣計算出獨立的來源訊號；非負矩陣分解 (Non-negative Matrix Factorization, NMF) [31] 為一種非監督式的機器學習技術，此方法是希望找到最好的兩個矩陣，使他們的乘積能接近原始的輸入。另外也常用主成分分析法 (Principal Component Analysis, PCA) [32] 來降低資料為度與消除雜訊。然而盲源分離的方法對於分離複雜度高的訊號，表現並不理想。

近年來隨著深度學習領域的蓬勃發展，大家漸漸將音訊資料的盲源分離變成一個子任務，也就是音源分離，目前最常見的目標為分離一首包含人聲、貝斯、鼓與其他 (非前三者的所有音訊) 的歌曲，音源分離的目標最常見的為分離一首包含人聲、貝斯、鼓與其他 (非前三者的所有音訊) 的歌曲，在大型音源分離比賽 [24], [25] 中也是使用此標準進行。音源分離的方法可分為基於時間頻率表示方法與基於波形表示方法，第一種通常會預測每個來源的功率頻譜，並結合輸入的混合音訊相位來合成每個音源的波形，大部分會使用全連接神經網路

(Fully-Connected Neural Network, FC)、長短期記憶模型 (Long Short-Term Memory, LSTM)、遞歸神經網路 (Recurrent Neural Networks, RNN) 等方法訓練頻譜圖。2018 年由 Stöter 等人所提出的 Open-Unmix [33] 模型在當時在公開資料集 MUSDB [34] 上表現最佳，此模型使用三層雙向 LSTM 對輸入預測一個遮罩，將混合音訊透過遮罩得到分離音源頻譜圖。隨後在 2022 年由 Luo Yi 等人所提出的 Band-Split RNN [35] 在 MUSDB 上取得最佳的成績，此模型將輸入的頻譜圖分割為預先定義好頻帶寬度的子帶頻譜圖，將子帶頻譜圖轉為相同維度的特徵後利用堆疊 RNN 交錯訓練，最後經過多層感知機 (Multilayer Perceptron, MLP) 生成遮罩並與混合輸入結合產生分離音源頻譜圖，至今仍是基於時間頻率訓練的開源模型中效果最佳的模型。Open-Unmix 與 Band-Split RNN 皆允許使用者自訂不同的分離目標，使的這兩個模型擁有很高的靈活性。

第二種基於波形訓練的方法最早提出的模型為 Wave-U-Net [36]，近年來較多人使用的 Demucs [3] 就是基於 Wave-U-Net 的架構開發的，Demucs 為 Encoder/Decoder 架構，內部由卷積 Encoder/Decoder 與雙向 LSTM 組成，Encoder 與 Decoder 使用 Skip U-net 連接，將混合立體音訊作為輸入，輸出每個來源的估計波型。2021 年 MDX 競賽 [24] 由 Meta AI 提出的冠軍模型 Hybrid Demucs [37] 更是將 Demucs 擴展為多域分析的架構，模型變為由波型、頻譜和共享層組成。2022 年 Meta AI 結合先前的結果提出了 Hybrid Transformers [38]，將 Encoder/Decoder 內部兩層替換為跨域 Transformer，使的模型可以接受異質資料，變成更靈活的架構，2023 年在 MUSDB 上取得最佳的成績，為目前所有開源模型中最佳的模型。

在古典樂器錄音的音源分離研究中，由於大部分混合音訊中的樂器皆為音高樂器 (例如：鋼琴、小提琴等等)，比起音高樂器與非音高樂器 (例如：鼓) 的音源分離情況更有挑戰性，且公開資料集的蒐集難度高，導致訓練模型的難度也更大。因此針對這些資料通常會使用資料增強的方法來增加訓練資料，由 Miron 等人提出的 CNN-based 低延遲單

聲道源分離模型 [39]，透過改變速度、音色、動態時間局部變化來增加新的訓練資料，使模型在實際情況下更能應對演奏時的狀況。由 Chiu Ching Yu 等人提出針對資料規模小的資料集的資料增強方法 [40]，使用 Open-Unmix 模型作為架構提出 Aug4mss 模型，並在針對小提琴與鋼琴的混合音源分離的任務上，比當時可用的最佳模型 Spleeter [41] 的效果要好，目標為小提琴時在 SDR 指標上約提升了 1.28DB，鋼琴則提升了 4.46DB。

本研究將使用靈活性高且較輕量的 Band-Split RNN [35] 模型，針對小提琴與鋼琴的混合音源資料做訓練，旨在促進古典樂器錄音的音源分離研究進展。

2.2.2 音樂追蹤相關研究

音樂追蹤在 MIR 領域中是一項重要的應用，與節拍追蹤 [42], [43]、樂譜追蹤 [44]、自動伴奏 [8] 等相關技術有密切的關係，[42] 提出了線上節拍追蹤模型，使用特徵提取的單向 RNN 預測節拍函數與線上決策的粒子濾波加快決策過程；[43] 提出將尺度不變的卷積層應用於 CNN 的重拍跟蹤的重拍追蹤模型；[44] 使用強化式學習讀取音訊與樂譜影像學習如何追蹤目前音樂的位置；[8] 設計透過多基頻估計獲得訓練特徵的基於神經網路之鋼琴自動伴奏系統；本節專注探討以音訊特徵為主的即時音樂追蹤方法與應用的相關研究。

音樂追蹤的方法包含隱藏馬爾科夫模型 (Hidden Markov Model, HMM) [45]，HMM 透過特徵觀察每個狀態的機率迭帶訓練更新目前追蹤的位置，但由於 HMM 依賴於離散空間來表示樂譜的音符與節奏，因此不適合處理複雜的連續變化，且因為計算複雜性高，在即時表現方面較為遜色；動態時間規整 (Dynamic Time Warping, DTW) [46], [47]，DTW 透過計算兩段相似序列的成本矩陣，回推兩段音訊的最佳對齊點，由於音樂的連續性使的 DTW 計算結果準確率高，但也因為計算時間複雜度

高而無法應用在即時的音樂追蹤。因此許多人對 DTW 演算法改良了更快速的版本，例如平行化動態時間規整 (Parallel Dynamic Time Warping, PDTW) [48], [49]，PDTW 將兩個時間序列分割成多段的序列，每段序列各自計算 DTW，計算完畢後再將每段序列連接回整段，加速 DTW 的運算時間；線上動態時間規整 (Online Dynamic Time Warping, ODTW) [50]–[52] 將原本 DTW 向後對齊的模式改為增量對齊，並限制了搜索範圍，ODTW 不需要事先取得兩段序列的資訊，而是根據每個時間點計算出的路徑決定下一步的方向，因此 ODTW 在即時音訊的處理的情況會比前面提到的方法還要快速，更能達到即時音樂追蹤的效果，但也因為無法確定哪個位置為音訊結束的位置，因此通常追蹤的準確率會比 DTW 要低。

音訊特徵的音樂追蹤比起使用 MIDI 檔案的音樂追蹤要困難許多，因為追蹤結果對特徵的變化非常敏感，因此使用的音訊特徵通常越相像越好，例如使用同一人事先演奏的音訊來追蹤同一人的現場演奏，但通常使用者並不會特意去錄製事先演奏的音訊，尤其是在沒有合奏者的情況下，因此本研究致力於研究使用不同音訊特徵來追蹤現場演奏的即時音樂追蹤系統。本研究將基於 [52] 所提出的架構進行重新實現並針對 ODTW 演算法與部分方法進行改動，希望能藉此達到我們所期望的效果並促進即時音樂追蹤研究進展。

三、研究方法

3.1 系統架構

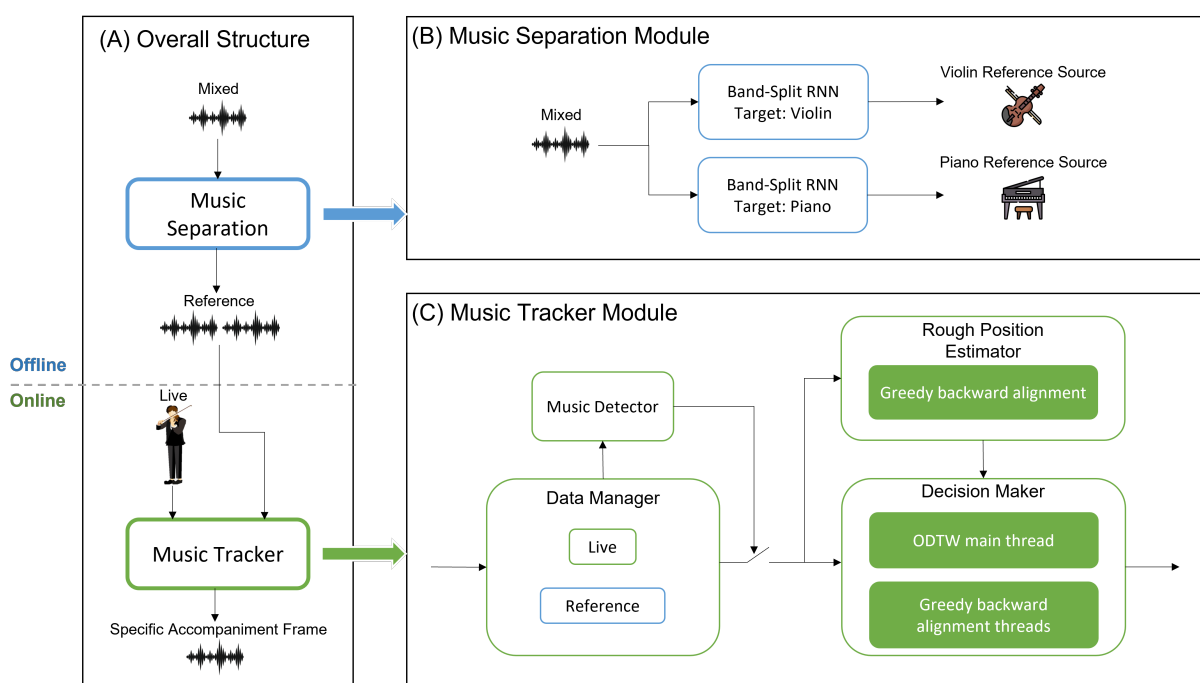


圖 3.1: 系統架構圖

圖 3.1 (A) 為本系統的整體架構，此系統可分為離線處理與線上處理兩個階段，離線處理階段會將混合音源分離成小提琴參考音訊與鋼琴參考音訊作為線上處理階段的資料，線上處理階段接收現場音訊，並結合參考音訊來計算目前現場音訊對應到的參考音訊位置並輸出估計伴奏音訊。請注意本研究將小提琴與鋼琴之混合音訊作為主要資料，並專注在當演奏者樂器為小提琴或鋼琴的情況下來使用本系統。

圖 3.1 (B) 為音源分離模組的細部架構，採用深度學習模型將小提琴

與鋼琴的混合音訊分離為小提琴參考音訊與鋼琴參考音訊。此模組包含了小提琴音源分離模型與鋼琴音源分離模型，另外在訓練階段，本研究設計了頻帶切割估計方法提升模型的分離效果。關於模型架構、頻帶切割估計方法會在3.2節做討論。

圖 3.1 (C) 為音樂追蹤模組的細部架構，我們將音源分離模組分離出來的參考音訊作為音樂追蹤模組的資訊，此模組將會即時接收現場音訊並使用小提琴參考音訊來對齊現場音訊，進而獲得目前現場音訊演奏的位置，由於小提琴參考音訊與鋼琴參考音訊為完美對齊的，我們可以透過目前對齊到的小提琴參考音訊位置取得目前鋼琴參考音訊的位置做為估計伴奏音訊輸出，達成即時追蹤與伴奏的效果。此模組使用 Python 3.9 開發，我們使用 multiprocessing package [53] 實現每個元件的多行程處理 (Multi-processing)，現場音訊使用 pyaudio package [54] 接收處理。此模組包含四個元件，分別為資料管理者 (Data Manager)、音樂偵測器 (Music Detector)、粗略位置估計器 (Rough Position Estimator) 與決策決定者 (Decision Maker)。資料管理者管理所有音訊的資料與特徵、音樂偵測器即時偵測音樂是否開始、粗略位置估計器估計整段小提琴參考音訊中目前現場可能的演奏位置，提供決策決定者更多選擇、決策決定者即時計算目前輸出位置並決定最後的輸出位置，關於元件詳細的設計與改動會在3.3節做討論。

3.2 音源分離模組

3.2.1 Band-Split RNN

Band-Split RNN(BSRNN) [35] 由 Luo Yi 等人於 2022 年提出，是一種專門為高取樣率訊號所設計的時頻域音源分離模型。BSRNN 的輸入為頻譜圖 $X \in \mathbb{C}^{F \times T}$ ，如圖 3.2 所示。

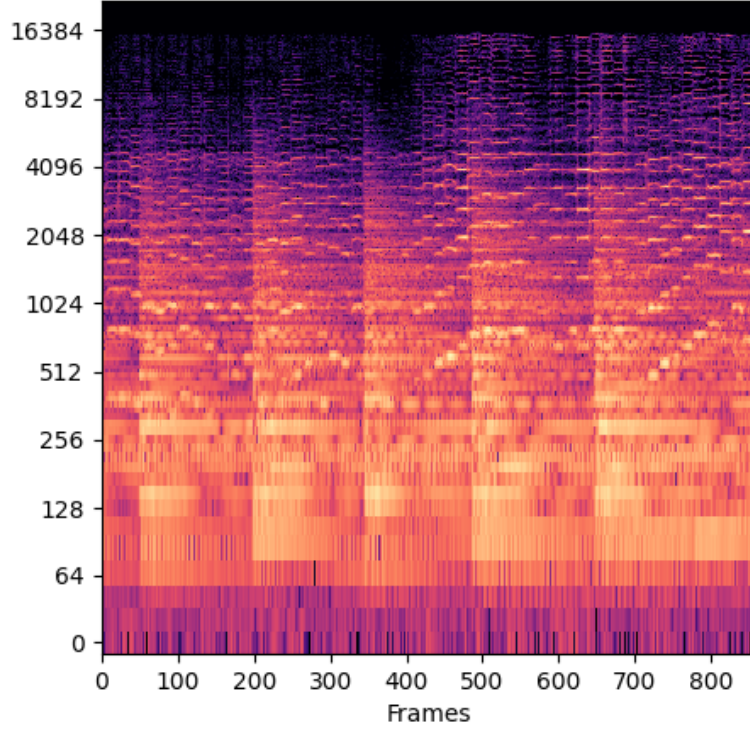


圖 3.2: BSRNN 輸入頻譜圖

接著會根據自定義的頻帶切割點 $G = (G_1, G_2, \dots, G_i)$, $\sum_{i=1}^K G_i = F$ 拆分成一個個的子帶頻譜圖 $B = (B_1, B_2, \dots, B_K)$, $B_K \in \mathbb{C}^{G_K \times T}$ ，每個不同大小的子帶頻譜圖經過處理後變為一系列具有相同維度的特徵 $Z \in \mathbb{R}^{N \times K \times T}$ ，接著利用 RNN 層將頻帶與序列的資訊交錯建模得到輸出 $Q \in \mathbb{R}^{N \times K \times T}$ ，最後每個子帶特徵由 MLP 產生對應的複值時頻遮罩 $M \in \mathbb{C}^{F \times T}$ ，並將遮罩用於輸入的混合頻譜圖來產生估計的目標源頻譜

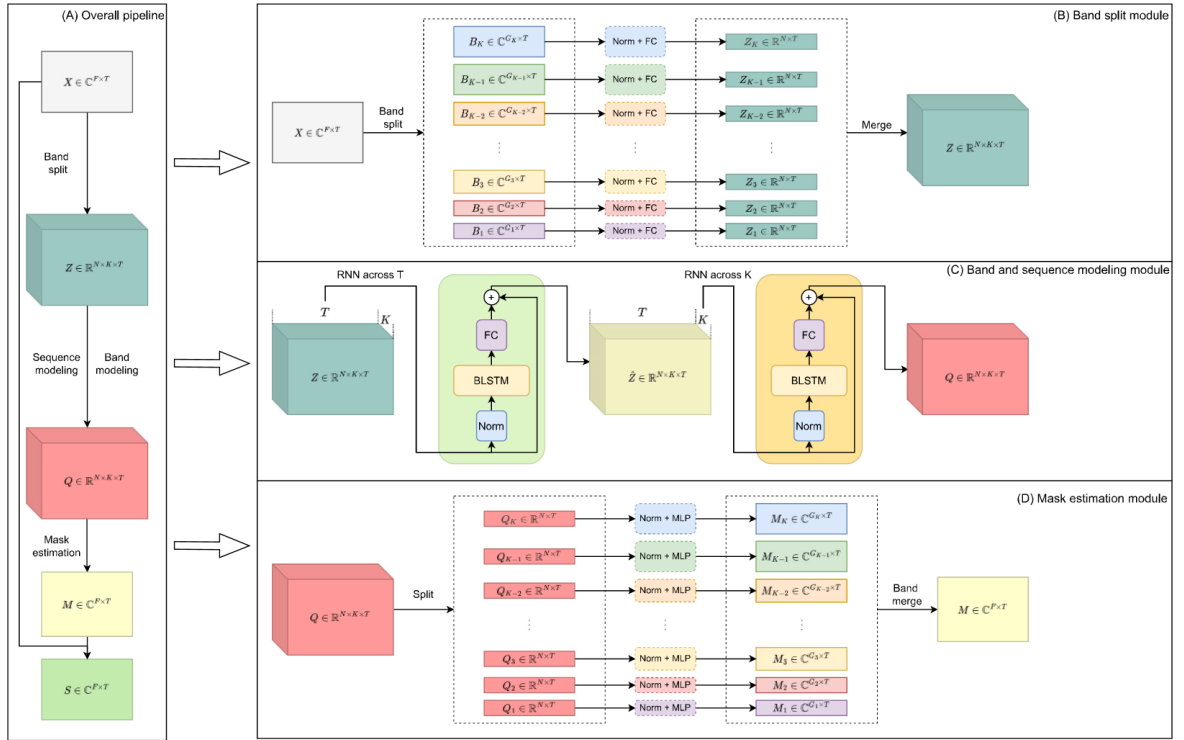


圖 3.3: BSRNN 架構圖，擷取自 [35]

圖 $S \in \mathbb{C}^{F \times T}$ 。整體架構如圖 3.3 所示。

其中在圖 3.3 (B) 的頻帶切割模組 (Band split module) 是 BSRNN 模型中最重要的模組，此模組可根據目標音源的聲音特性來設計不同的頻帶切割範圍，例如當目標音源的聲音特性都集中在較低頻的地方時，我們可以將低頻切割得更細，高頻切割的寬一點，使模型在訓練時可以看到更重要的頻域特徵。在 [35] 中頻帶切割的位置為事先透過專家或先驗知識定義好的。為了更方便估計不同樂器的頻帶切割範圍，我們在章節 3.2.2 提出頻帶切割估計方法。

在 BSRNN 的論文中，他們將模型應用於分離人聲、貝斯、鼓與其他這四種目標音源，我們將訓練目標改為分離小提琴與鋼琴這兩種音源，來達成本研究的目的。

3.2.2 頻帶切割估計方法

為了使 Band-Split RNN 中的頻帶切割模組發揮更好的效果，我們使用簡單的頻帶切割估計方法來估計小提琴與鋼琴的頻帶切割點，此方法不需要依賴先驗知識或專家知識的幫助，而是透過同一種樂器的音訊資料來估計頻帶切割點。首先我們對每種樂器選取約 15 分鐘的音訊資料，對於小提琴資料，我們分別選取有不同速度與不同難度的技巧(例如：雙音、跳弓等)表現的音訊，而鋼琴資料則是選取有不同速度的音訊，這些資料並不會加入模型的訓練。對於每筆資料，使用取樣率 $sr = 44100\text{Hz}$ 讀取，並初始化頻帶陣列 $F_{band} \in \frac{sr}{2}$ 來記錄每個頻率占有的成分多寡。使用快速傅立葉轉換 (Fast Fourier Transform, FFT) 計算頻率域的每個頻率樣本占有的成分，由於 FFT 的對稱性，我們只需要迭帶一半的樣本數，並將振幅值累加到對應的 F_{band} 。累加振幅值越高代表此頻率的重要程度越高，如圖 3.4 所示，此圖為兩種樂器中其中一段音訊資料的波形圖 (上) 與經過 FFT 轉換後的 F_{band} 圖 (下)，可以看到經過 FFT 轉換後我們可以清楚的看到每個頻率點在這段音訊中佔有的成分。

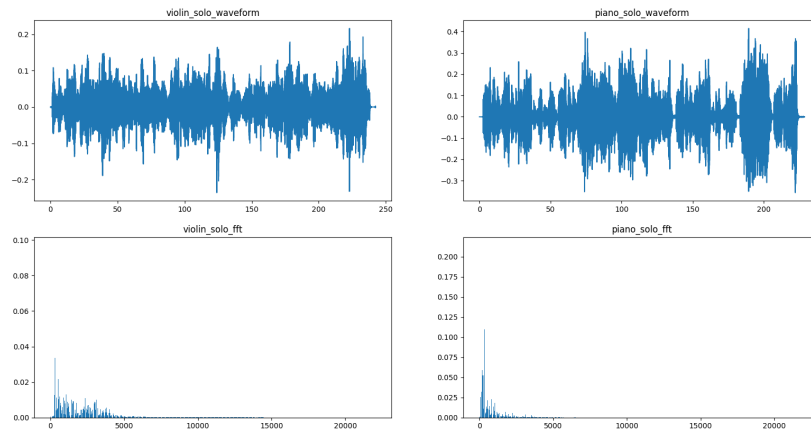


圖 3.4: 兩種資料的波形圖 (上) 與經過 FFT 轉換後的 F_{band} 圖 (下)

接著我們定義估計方法的一些參數，如表 3.1 所示。切割比例閥值控制切割點的設置，最小頻率切割值確保每個切割點都有足夠的頻率寬度，以免特徵過小，預計切割頻帶數量控制模型訓練時拿到的子頻帶數量，

上述這些參數皆可根據硬體的算力做調整。其中切割比例閾值 r_{violin} 與 r_{piano} 的值是根據預計切割頻帶數量所實驗出來的數值。

表 3.1: 頻帶切割估計方法參數設定

	Violin	Piano
切割比例閾值 r	$r_{violin} = 0.0016$	$r_{piano} = 0.0054$
最小頻率切割值 f_{min}	100Hz	
預計切割頻帶數量 c	40	

最後我們對 F_{band} 的範圍迭帶計算每個區間的頻率成份佔總成份的累積百分比 $f_{range} / \sum F_{band}$ ，若超過切割比例閾值 r ，則在此頻率點增加一個切割點。依照上述的切割比例我們得到兩種樂器的頻帶切割點為

Violin = [355,455,555,655,755,855,955,1055,1155,1255,1396,1515,1639,1763,1863,1963,2063,2212,2312,2412,2512,2629,2735,2840,2964,3079,3211,3383,3590,3795,3986,4205,4512,4923,5486,6314,7570,9488,13042]

Piano = [100,200,300,400,500,600,700,800,900,1000,1100,1200,1300,1400,1500,1600,1700,1800,1900,2000,2100,2200,2300,2400,2511,2627,2727,2827,2966,3114,3238,3422,3624,3975,4281,4773,5417,6591,9191]

以小提琴的頻帶切割點來看，訓練時所使用的子頻帶為 (0-355Hz, 355-455Hz, ... , 13042-22050Hz) 共 40 個，鋼琴的子頻帶為 (0-100Hz, 100-200Hz, ... , 9191-22050Hz) 共 40 個，在訓練 BSRNN 模型時我們將會套用上面兩個頻帶切割範圍做訓練。

3.3 音樂追蹤模組

此模組是參考 [52] 的 Real-Time Music Tracker 來重新實現，本節將分成兩個部分說明，3.3.1 節至 3.3.3 節會介紹實現音樂追蹤模組所使用到的演算法並詳細說明改動的部分，3.3.4 節至 3.3.7 節會介紹每個元件的設計與改動。

3.3.1 動態時間規整 (Dynamic Time Warping, DTW)

DTW [46]，是一種用來計算兩個時間序列資料之間相似度的方法。給定兩個時間序列 $X = (x_1, x_2, \dots, x_N)$, $N \in \mathbb{N}$ 和 $Y = (y_1, y_2, \dots, y_M)$, $M \in \mathbb{N}$ ，其中 N 和 M 為兩個時間序列的長度， x_N 和 y_M 是任兩個具有時間順序且形狀相似的資料，例如音樂、語音等等，如圖 3.5 所示。

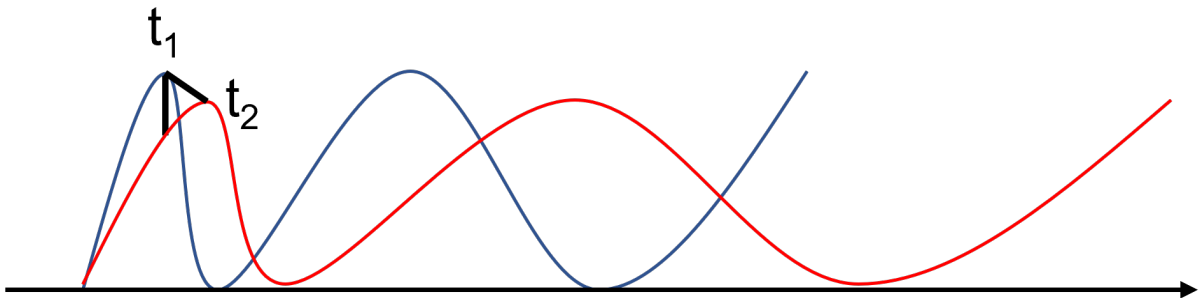


圖 3.5: 兩個相似的時間序列：藍色: X 紅色: Y

可以看到計算 $X[t_1]$ 與 $Y[t_1]$ 的距離，並不會是相似的資料點，但若計算 $X[t_1]$ 與 $Y[t_2]$ 的距離，就會被認為是兩個相近的資料點，因此當兩段相似的資料但在時間軸上並不是對齊的，就適合用 DTW 演算法動態對齊時間。考慮到時間序列的長度不一定相等的情況，例如不同人演奏同一個音符可能會有不同的速度差異，因此 DTW 通常採用動態規劃的方法來計算相似度。

DTW 的計算方式如下，首先使用歐基里德距離式 (3.1) 計算 X 與 Y

所有時間點的距離，如圖 3.6 所示。

$$d(i, j) = \sqrt{|x_i - y_j|^2} \quad (3.1)$$

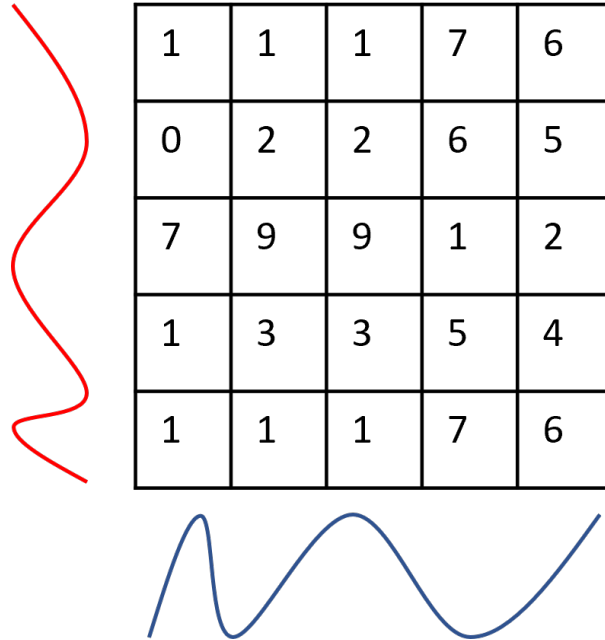


圖 3.6: DTW 單格成本距離矩陣示意圖

並根據式 (3.2) 使用動態規劃方式建立距離成本矩陣 $D \in \mathbb{R}^{N \times M}$ 。

$$\left\{ \begin{array}{l} D(0, 0) = d(x_1, y_1) \\ D(i, 0) = D(i - 1, 0) + d(x_i, y_1) \\ D(0, j) = D(0, j - 1) + d(x_1, y_j) \\ D(i, j) = d(x_i, y_j) + \min \left\{ \begin{array}{l} D(i, j - 1) \\ D(i - 1, j) \\ D(i - 1, j - 1) \end{array} \right\} \end{array} \right. \quad (3.2)$$

在計算最短路徑時，必須遵守三個限制：

1. 邊界限制：路徑的開頭與結束必須為 (x_1, y_1) 與 (x_N, y_M)

2. 連續性：若路徑的上一個對齊點為 (i, j) ，下一個對齊點只能是 $(i+1, j+1), (i+1, j), (i, j+1)$
3. 單調性：若路徑的上一個對齊點為 (i, j) ，下一個對齊點必須為 $(i_{next}, j_{next}) \in (i_{next} \geq i, j_{next} \geq j)$

這是為了使回溯的路徑為連續無倒退且整段序列都被包含在輸出裡，確保對齊的一致性。圖 3.7 為累積距離成本矩陣計算示意圖，計算完所有時間點的距離成本後 (如圖 3.7 左邊)，透過三個限制與式 (3.2)，獲得每個時間點的累積距離成本 (如圖 3.7 右邊)。

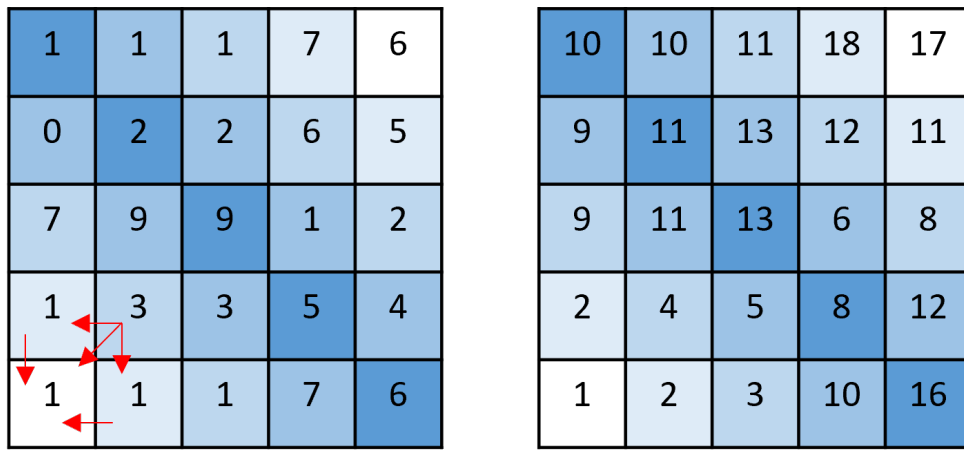


圖 3.7: DTW 累積距離成本矩陣計算示意圖

最後從 $D(N, M)$ 往前回溯至 $D(1, 1)$ 尋找一條最小的累積距離成本路徑，計算公式如式 (3.3) 所示，選擇回溯點時會確認行、列與對角三個方向的距離成本值，並選擇成本最小的點作為回溯點，

$$\begin{cases} (i_{next}, j_{next}) = \operatorname{argmin}_{(i-1, j-1), (i-1, j), (i, j-1)} D(i_{next}, j_{next}) \\ \text{Stop,} & \text{if } (i, j) = (1, 1) \end{cases} \quad (3.3)$$

當回溯點走到起始點 $(1, 1)$ 時，將所有回溯點連起來就得到最後對齊路徑 W ，如示意圖 3.8 所示，右上角為路徑的結束位置 $D(N, M)$ ，左下角為路徑的開始位置 $D(1, 1)$ ，綠色路徑即為 W 。

10	10	11	18	17
9	11	13	12	11
9	11	13	6	8
2	4	5	8	12
1	2	3	10	16

圖 3.8: DTW 累積距離成本矩陣回溯路徑示意圖

本研究將 DTW 演算法應用在音樂偵測器元件 (章節 3.3.5) 與即時音樂追蹤系統的評估上，我們使用 python librosa package [55] 中的 librosa.sequence.dtw 函式實現。

3.3.2 線上動態時間規整 (Online Dynamic Time Warping, ODTW)

由於 DTW 演算法在時間複雜度上為 $O(NM)$ ，屬於複雜度高的演算法，因此當某個序列的時間較長或是不確定序列的時間長度 (例如即時串流音訊) 時，DTW 演算法在計算就會花費許多時間。

ODTW [50] 由 Dixon 等人提出，是基於 DTW 演算法所做的改良，首先 ODTW 在計算對齊路徑時並不是回溯計算而是向前增量計算，並且將計算距離成本的範圍縮小至預先定義的常數範圍 c ，因此使 ODTW 在時間與空間複雜度的表現都是線性 $O(N)$ ，且因為不需事先知道整個序列的資料，因此適合處理長序列音訊或即時串流音訊。計算路徑範例如圖 3.9 所示。

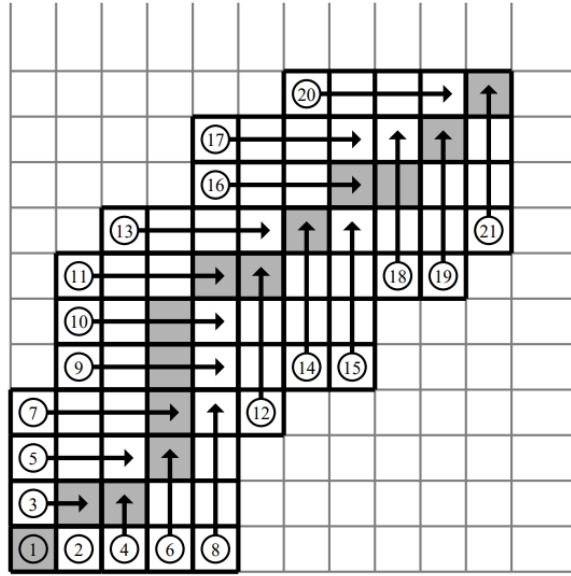


Figure 2: An example of the on-line time warping algorithm with search window $c = 4$, showing the order of evaluation for a particular sequence of row and column increments. The axes represent the variables t and j (see Figure 1) respectively. All calculated cells are framed in bold, and the optimal path is coloured grey.

圖 3.9: ODTW 對齊路徑計算，擷取自 [50]

我們將 [50] 提出的 ODTW 演算法稱為傳統 ODTW 演算法，並說明本研究改動的部分。本研究實現 ODTW 演算法的步驟如下，給定兩個時間序列 $X = (x_1, x_2, \dots, x_N)$, $N \in \mathbb{N}$ 和 $Y = (y_1, y_2, \dots, y_M)$, $M \in \mathbb{N}$ ，其中 X 為現場音訊，因此 N 會隨著串流時間長度而增加，直到串流結束， Y 為事先給定的參考音訊，用於對齊現場音訊。我們想透過累積距離成本矩陣 D_{total} 找到 X 和 Y 的對齊路徑 $W = (W_1, W_2, \dots, W_k)$ ，其中每個 W_k 為有順序的 (i_k, j_k) 且 $(i, j) \in W$ 代表兩段序列在 x_i 與 y_j 點上對齊。

計算累積距離成本矩陣 D_{total} 的方式使用 3.3.1 節的式 (3.1) 與式 (3.2) 計算，並且也記錄單格距離成本矩陣 D_{cell} 。

傳統 ODTW 演算法在計算兩種距離成本矩陣時，會根據目前的決定的方向 (Row、Column 或 Both) 決定計算的成本格數，關於如何決定方向會在後面作討論，我們先預設目前方向為 Both，也就是計算新的 $i(\text{Column})$ 與新的 $j(\text{Row})$ 的成本矩陣，計算方式如下，當時間點固定為 i 時，計算 j 方向的距離成本 $d(i, j - c + 1), d(i, j - c + 2) \dots d(i, j + 1)$ ；

當時間點固定為 j 時，計算 i 方向的距離成本 $d(i - c + 1, j), d(i - c + 2, j) \dots d(i + 1, j)$ ，在兩個方向上分別計算 c 格距離成本。

由於本研究的即時音樂追蹤系統在現場音訊的時間點是嚴格單調遞增，我們想在現場音訊時間點為 i 時能取得更多參考音訊時間點的成本資訊，因此我們將時間點為 i 時所計算的 j 方向格數改為 $2c - 1$ ，當有新的 i 加入時，我們計算 $d(i, j - c + 1), d(i, j - c + 2) \dots d(i, j + c - 1)$ 個點，而對於 j 點，保持計算 $d(i - c + 1, j), d(i - c + 2, j) \dots d(i + 1, j)$ 。

兩種不同的距離成本矩陣計算方式如範例圖 3.10 所示，假設每次迭帶的路徑方向皆為 Both，且搜尋範圍 $c = 3$ ，我們與傳統 ODTW 不同之處為在 i 時間點時所看到的參考音訊點的數量多寡。

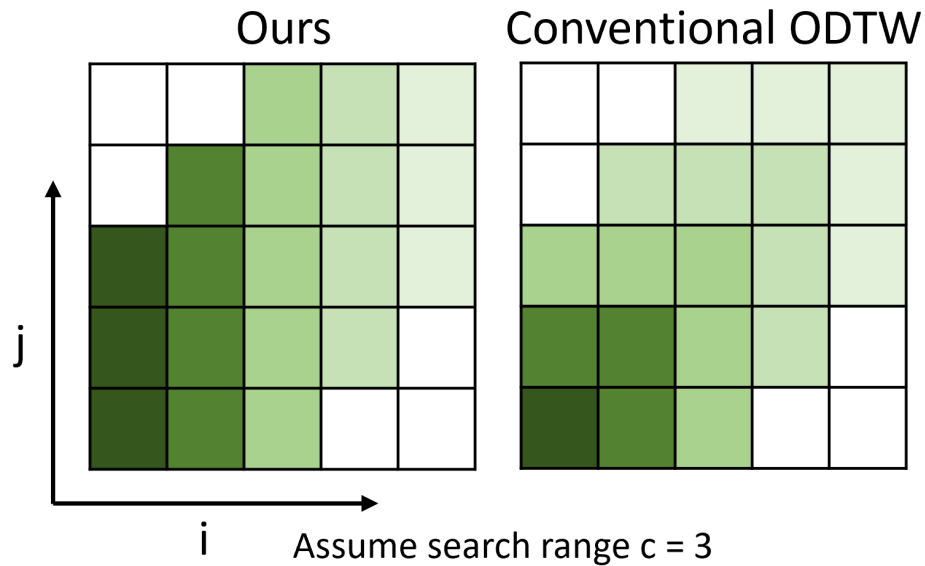


圖 3.10: 每次迭帶所計算到的距離成本矩陣格數位置比較

計算完最新的 D_{total} 與 D_{cell} 後，接著計算 ODTW 的輸出位置，在傳統 ODTW 中，他們根據式 (3.4) 計算 i 方向與 j 方向上累積距離成本最小的點作為輸出對齊點，計算範圍如圖 3.11 所示。

$$(output_i, output_j) = \underset{(i-c \leq x \leq i, y=j) \text{ and } (x=i, j-c \leq y \leq j)}{\operatorname{argmin}} D_{total}(x, y) \quad (3.4)$$

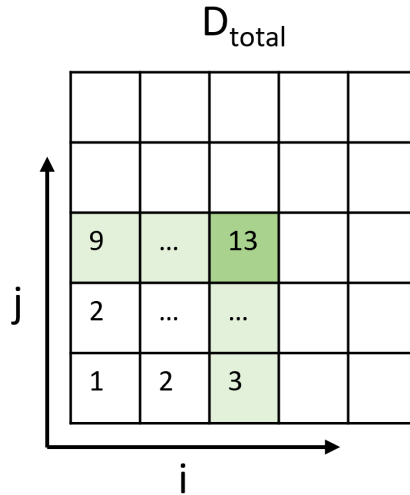


圖 3.11: 傳統 ODTW 計算輸出對齊點範圍示意圖

由於我們的 i 為現場串流音訊的時間點，並不會有回溯的情況發生，我們需要知道的是在 i 時間點時，參考音訊的對齊位置 $output_j$ ，因此我們將計算輸出對齊的方法稍作修改。首先我們設定兩個參數 L_{upper} 與 L_{lower} ，如表 3.2 所示，分別代表尋找 $output_j$ 的上下限。 L_{upper} 的設定為

表 3.2: ODTW 計算輸出範圍參數設定

L_{upper}	L_{lower}
$j + 30$	$j - 1$

可調整的，此數值大約允許現場音訊比參考音訊提前 0.6 秒，另外考慮到現場音訊並不會突然往前跳回前面的位置，只會有提前或落後的情況產生，因此我們的 L_{lower} 不考慮小於 j 的位置，若現場音訊目前落後，計算出來的輸出點就會停留在 j 點，以免輸出陷入局部最小值影響到後面的對齊。接著根據式 (3.5) 計算輸出對齊點 $(output_i, output_j)$ ，計算範圍如圖 3.12 所示。在現場音訊 $x = i$ 時，我們迭帶所有介於 L_{upper} 與 L_{lower} 的點 y ，找到一組 D_{total} 與 D_{cell} 相加為最小的點，此點就是 ODTW 的輸出對齊點，加入 D_{cell} 來計算是為了避免 D_{total} 的值誤判此點的相似度。

$$(output_i, output_j) = \underset{(x=i, L_{lower} < y \leq L_{upper})}{argmin} (D_{total}(x, y) + D_{cell}(x, y)) \quad (3.5)$$

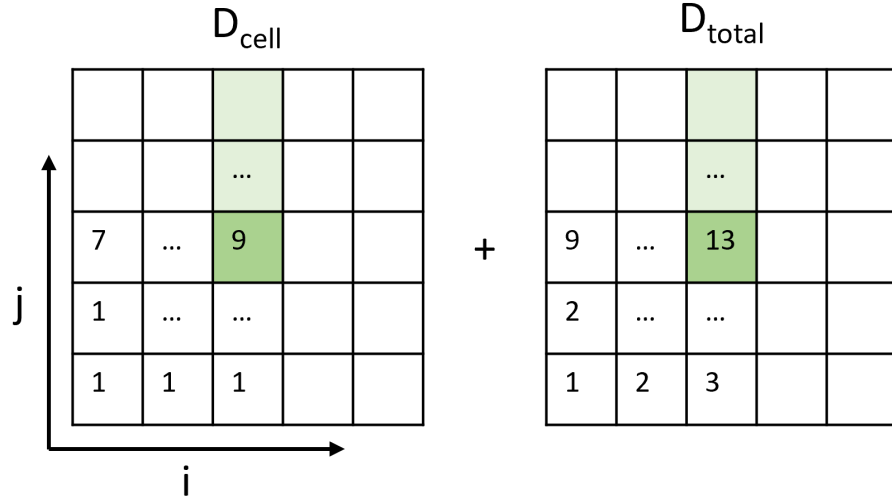
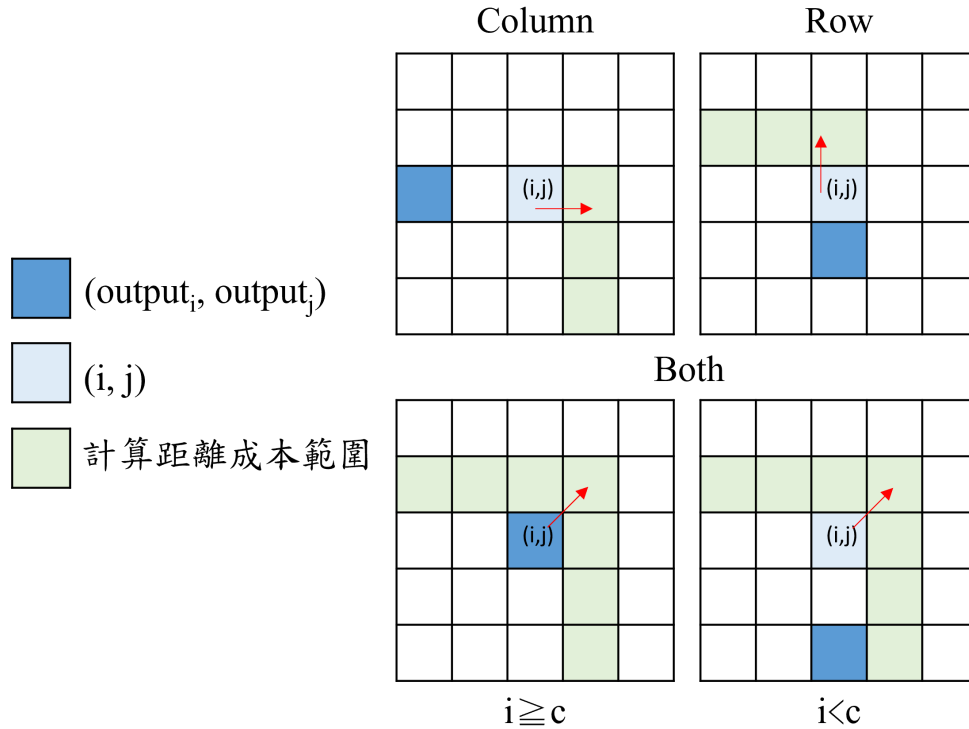


圖 3.12: 改動後 ODTW 計算輸出對齊點範圍示意圖

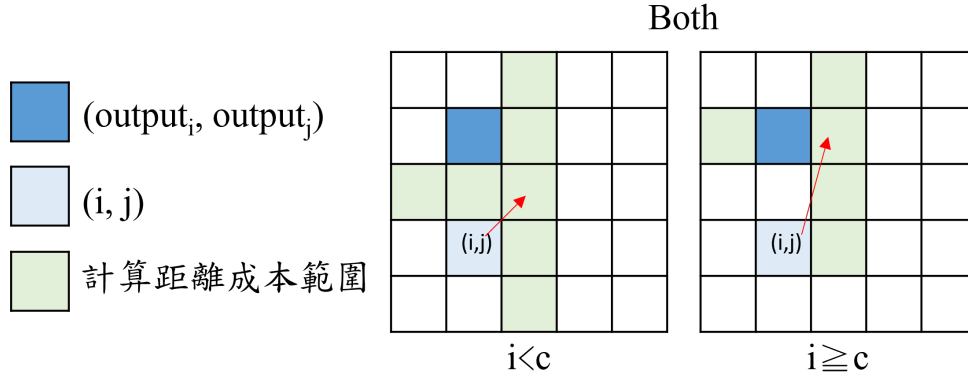
另外在計算完目前最小成本值後，為了避免系統陷入局部最小值的狀況，我們增加了跳躍機制，當 $output_j$ 的值連續 9 次 (約 0.18 秒) 都相同的話，我們強制將下一個 $output_j$ 往上加 1。

在計算完 $(output_i, output_j)$ 後，傳統的 ODTW 演算法會根據目前的 (i, j) 與輸出的 $(output_i, output_j)$ 決定下一步的走向 (i_{next}, j_{next}) 以增加新的距離成本到兩個矩陣中，走向分別有 $Both(i+1, j+1)$ 、 $Column(i+1, j)$ 、 $Row(i, j+1)$ 三種，如圖 3.13 所示，紅色箭頭所指的地方為 (i_{next}, j_{next}) 。

圖 3.13: 傳統 ODTW 決定計算方向示意圖, $c = 3$

首先當 $i < c$ 時，為了使搜索範圍內的隔數都有計算到距離成本，因此計算 Both 方向 (圖 3.13 右下角)，當 $i \geq c$ 後，開始根據輸出的位置判斷走向，當 $output_i < i$ 時，代表目前 i 方向輸出的點較慢，因此往方向 Column 計算距離成本 (圖 3.13 左上角)；當 $output_j < j$ 時，代表目前 j 方向輸出的點較慢，因此往方向 Row 計算距離成本 (圖 3.13 右上角)；若目前 $(output_i, output_j)$ 皆與 (i, j) 對齊，繼續往 Both 方向計算新的距離成本 (圖 3.13 左下角)。

由於我們的 i 方向為現場音訊的時間點， $output_i$ 並不會輸出 $i + 1$ 以外的輸出點 (嚴格單調遞增)，且輸出 $output_j$ 並不是連續的，傳統 ODTW 的方法不適用於我們改動的輸出機制，因此我們將增量對齊的方向改為全部採用 Both 的方向計算，但當 $i \geq c$ 後，我們將目前的 j 設為 $output_j$ ，使系統目前的 j 點保持與輸出相同的位置。如圖 3.14 所示。

圖 3.14: 改動後 ODTW 決定計算方向示意圖, $c = 3$

3.3.3 貪心向後對齊 (Greedy Backward Alignment, GBA)

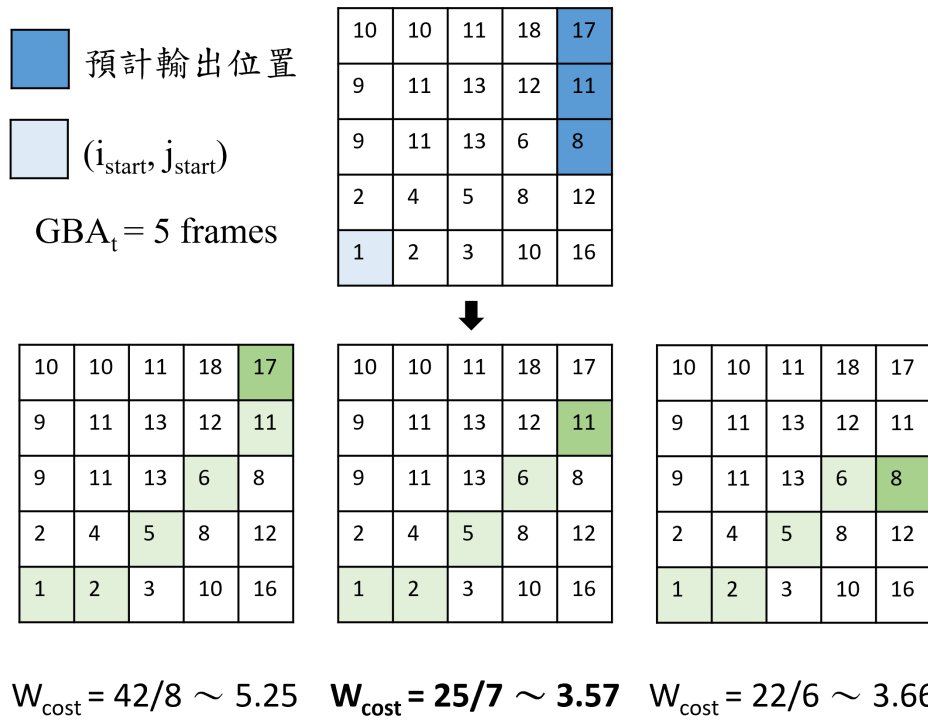
由於 ODTW 在對齊時為增量對齊，並不像 DTW 一樣在計算完兩段音訊的距離成本矩陣後才回溯最佳對齊路徑，因此通常 ODTW 的輸出結果會比 DTW 的輸出結果來的更不穩定。為了改善上述問題，我們在追蹤啟動後一段時間加入此方法輔助系統計算輸出位置，我們參考 [51] 中的 Greedy Local Search 演算法來設計此方法，此方法配合多執行續的執行可以快速的計算目前位置的向後對齊成本。

給定一個預計輸出位置 (i, j) 作為向後對齊的結尾點，此位置代表在現場音訊為 i 時，參考音訊估計的對齊位置為 j ，我們設定一個向後對齊距離參數 GBA_t ，此參數代表向後對齊的時間長度。根據現在 i 的時間點與先前輸出位置紀錄 W_{output} ，我們可以推算出向後對齊的開頭點 $(i_{start}, j_{start}) = W_{output}[i - GBA_t]$ 。

接著根據累積距離成本矩陣 D_{total} ，我們可以透過 3.3.1 節的式 (3.3) 計算 (i_{start}, j_{start}) 到 (i, j) 的對齊成本 W_{cost} ，在回溯的過程中，因為 (i_{start}, j_{start}) 與 (i, j) 並不是真正的起始與結束點，我們設定了最大步數 $GBA_{Max_Run} = 3$ 來限制回溯路徑的走向，若過程中在行或列的方向走超過 GBA_{Max_Run} ，下一步只計算其他兩種方向的對齊成本決定走向。另外在計算累積距離成本時，我們紀錄回溯路徑的總步數 $step_{total}$ ，並針對

三種走向設計不同大小的值，當走向為對角時， $step_{total}$ 增加 2 步，當走向為行或列時， $step_{total}$ 增加 1 步。這是為了正規化總累積成本，使每個預計輸出位置在比較累積距離成本時能公平的比較，最後對齊成本計算公式如式 (3.6) 所示。範例圖如圖 3.15 所示。

$$W_{cost} = \frac{\sum_{w=(i_{start}, j_{start})}^{(i, j)} D_{total}(w)}{step_{total}} \quad (3.6)$$

圖 3.15: GBA 計算 W_{cost} 示意圖

3.3.4 資料管理者元件 (Data Manager Block)

在線上處理階段啟動前與啟動後，我們對非串流音訊 (參考音訊) 與串流音訊 (現場音訊) 擷取兩種特徵作為其他元件的輸入資料，分別為高解析度特徵與低解析度特徵，高解析度特徵為梅爾頻譜特徵 (Mel Spectrogram)[56]，低解析度特徵為高解析度特徵對時間軸卷積後的重新採樣。為了管理音訊特徵的處理與傳送，我們設計了資料管理者來集中

管理音樂追蹤模組中所有用到的音訊特徵。

在啟動追蹤系統前，資料管理者會拿到音源分離模組分離出來的小提琴參考音訊與鋼琴參考音訊，對於這兩段完整的音訊，使用 44100Hz 的取樣率讀取，並計算短時距傅立葉轉換 (Short-time Fourier Transform, STFT) 得到頻譜圖，STFT 的參數設定如表 3.3 所示。

表 3.3: STFT 參數設定

Window size	Hop size	Number of fft	Window function
46ms (~ 2028 frames)	20ms (~ 882 frames)	46ms (~ 2028 frames)	Hanning

為了讓用來對齊現場音訊的小提琴參考音訊特徵更接近人類的聽覺系統，我們對小提琴參考音訊的 STFT 頻譜圖做梅爾頻譜分析，得到高解析度特徵。梅爾頻譜是根據人耳對聲音的敏感度所設計的一種時頻分析方式，而梅爾頻譜是由梅爾濾波器組 (Mel Filter Bank) 與 STFT 頻譜圖矩陣相乘後得到的結果，梅爾頻譜圖的參數設定如下表 3.4 所示， n_mel 為濾波器數量， f_{min} 與 f_{max} 為使用率波器的頻率範圍，代表梅爾頻譜特徵在 0-8000 Hz 使用了 84 個濾波器將此頻率範圍壓縮成更接近人耳所聽到的樣子。

表 3.4: 高解析度特徵參數設定

n_mels	f_{min}	f_{max}
84	0 Hz	8000 Hz

低解析度特徵由前面所提到的高解析度特徵對時間軸進行卷積，參數設定如表 3.5 所示，將每個 Window size 的高解析度特徵窗與 Window function 做卷積並取平均值代表此低解析度特徵時間點的頻率份量。

表 3.5: 低解析度特徵參數設定

Window size	Hop size	Window function
600ms (~ 30 frames)	300ms (~ 15 frames)	Hanning

追蹤系統啟動後，現場音訊以單聲道、樣本數為 Window size 的大小傳送到系統，對於現場音訊的特徵處理方法與上述相同，但由於在接

收音訊時並不像非串流音訊一樣直接拿到整段音訊，因此必須確保接收到足夠的樣本數才能擷取特徵，否則可能導致特徵蒐集不完全而失真。因此當系統啟動後，我們將現場音訊記錄下來，當音訊累積足夠的長度時再擷取高/低解析度特徵。

圖 3.16 為現場音訊與小提琴參考音訊之特徵圖，可以看到高解析度特徵的時間軸切片較細，低解析度特徵時間軸切片較粗。另外現場音訊雖然是以 Window size 大小處理特徵，但當追蹤結束時，現場音訊特徵與使用整段音訊做特徵擷取的小提琴參考音訊特徵，在視覺化上並沒有特別明顯的差異（無特徵失真的現象）。

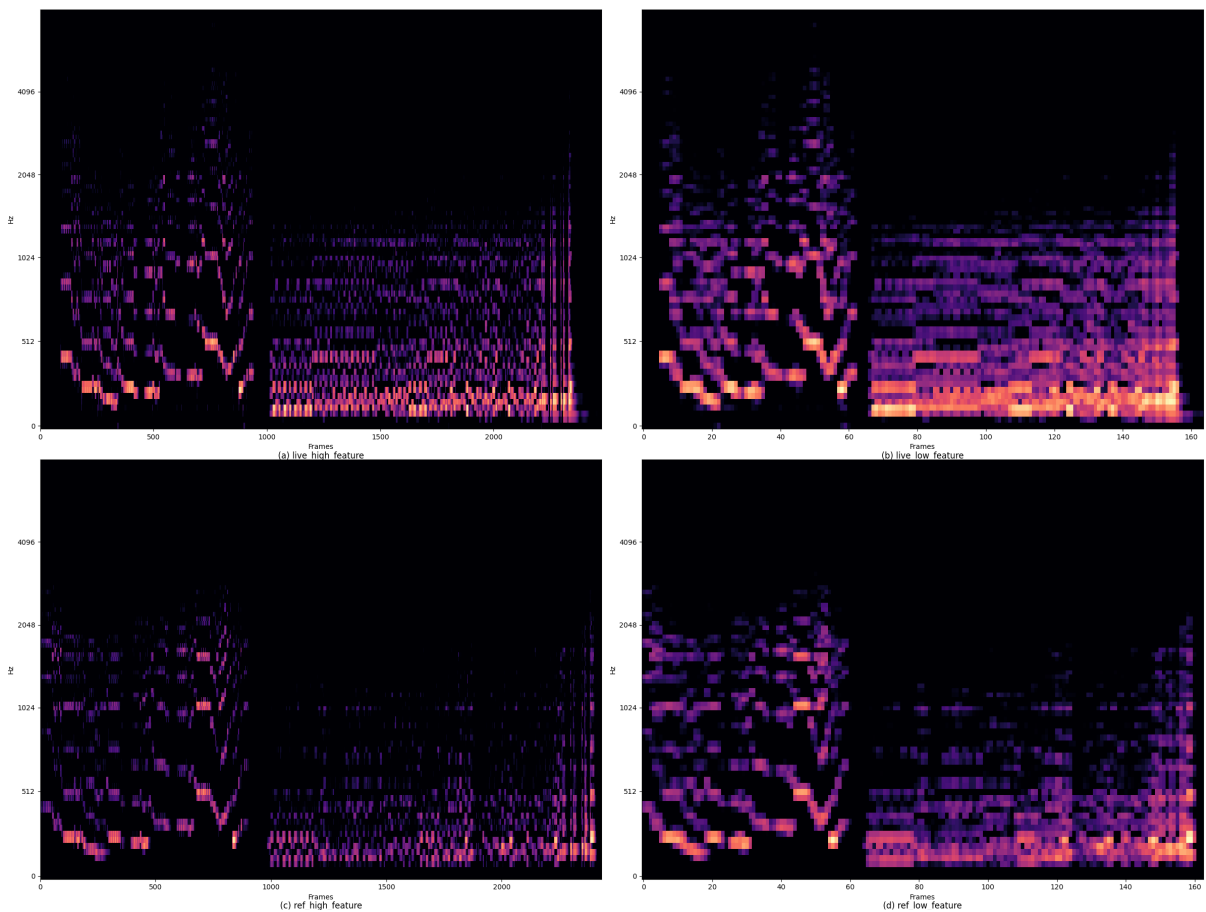


圖 3.16: 現場與參考的高低解析度特徵圖：(a) 現場高解析度，(b) 現場低解析度，(c) 參考高解析度，(d) 參考低解析度

3.3.5 音樂偵測器元件 (Music Detector Block)

音樂偵測器負責偵測使用者開始演奏的時間並觸發後面元件執行，我們實作了平均振幅的閾值判斷並參考 [52] 的 DTW 對齊成本閾值判斷方法來實現。

當音樂偵測器接收到現場音訊時，我們計算平均振幅並根據事先定義的閾值 θ_A 判斷此片段是否為靜音片段，振幅平均值的計算公式如式 (3.7) 所示， N 為整段音訊的樣本數， $|x[n]|$ 為音訊第 n 個樣本的絕對值， A 為振幅平均值。

$$A = \frac{1}{N} \sum_{n=1}^N |x[n]| \quad (3.7)$$

θ_A 是透過統計的方式來決定設定的值，我們使用已標記音樂開始時間的音訊來一一測試，已標記的音訊是使用自己錄製的音檔來測試，我們透過測量每一首的音樂開始時間的振幅值與音樂尚未開始時間的振幅值，並將音樂開始時間的振幅值加總後取平均獲得，最後我們將 θ_A 設為 0.01。

式 (3.8) 為平均振幅與閾值的條件判斷公式，當 $A \leq \theta_A$ 時，代表此片段為靜音片段，因此不考慮音樂開始的可能性，接著計算下一段音訊的平均振幅，當 $A > \theta_A$ 時，代表此片段為非靜音片段，可能為音樂開始的地方，因此將此片段接著使用 DTW 對齊成本閾值判斷方法計算。

$$\begin{cases} \text{Compute DTW cost,} & \text{if } A > \theta_A \\ \text{Do nothing,} & \text{if } A \leq \theta_A \end{cases} \quad (3.8)$$

另外平均振幅閾值判斷方法也用在最後演奏完畢的判斷上，若超過 3 秒以上為靜音片段則判斷為演奏結束。

DTW 對齊成本閾值判斷方法的實現如下，當現場音訊通過平均振幅閾值的判斷後，我們將現場音訊擷取為高解析度特徵並與小提琴參考音訊開頭 0.5 秒的高解析度特徵計算 DTW 對齊成本 DTW_{cost} 。若 DTW_{cost}

小於設定好的閾值 θ_{cost} ，音樂偵測器就會觸發追蹤事件 (觸發粗略估計位置與決策決定者元件)。 θ_{cost} 與上述提到 θ_A 的設定方法一樣為統計的方式，我們將 θ_{cost} 設為 2000。

式 (3.9) 為 DTW 對齊成本與閾值的條件判斷公式，當 $DTW_{cost} \geq \theta_{cost}$ 時，認定此兩段音訊的相似度不足，因此不觸發後面的元件並接著計算下一段音訊的平均振幅，當 $DTW_{cost} < \theta_{cost}$ 時，認定此兩段音訊具有足夠的相似度被認定為音樂開始的片段，因此觸發後面的元件並關閉自己的 process。

$$\begin{cases} \text{Trigger tracking event,} & \text{if } DTW_{cost} < \theta_{cost} \\ \text{Do nothing,} & \text{if } DTW_{cost} \geq \theta_{cost} \end{cases} \quad (3.9)$$

3.3.6 粗略位置估計器元件 (Rough Position Estimator Block, RPE)

在追蹤系統啟動後，為了避免決策決定者的錯誤對齊，粗略位置估計器負責使用低解析度特徵計算目前最新現場音訊幀與小提琴參考音訊的相似度，並挑出相似度高的前 8 個粗略估計位置提供給決策決定者參考。

當新的現場音訊幀經過特徵擷取後，粗略位置估計器會獲得新的現場低解析度特徵 $Live_{low}[t] \in \mathbb{R}^{1 \times F}$ ， t 為最新的低解析度特徵時間點。此時粗略位置估計器會計算 $Live_{low}[t]$ 與整個小提琴參考低解析度特徵 $Ref_{low} \in \mathbb{R}^{N \times F}$ 的歐基里德距離並紀錄到距離成本矩陣 $D_{RPE} \in \mathbb{R}^{t \times N}$ ， N 為 Ref_{low} 的時間長度。式 (3.10) 表示在現場與參考的時間點為 $[t, n]$ 時兩個低解析度特徵的距離。圖 3.17 為距離成本矩陣 D_{RPE} 的熱力圖，越

亮的地方代表兩格特徵距離越遠。

$$D_{RPE}[t, n] = \sum_{f=0}^F \sqrt{|Ref_{low}[n, f] - Live_{low}[t, f]|^2} \quad (3.10)$$

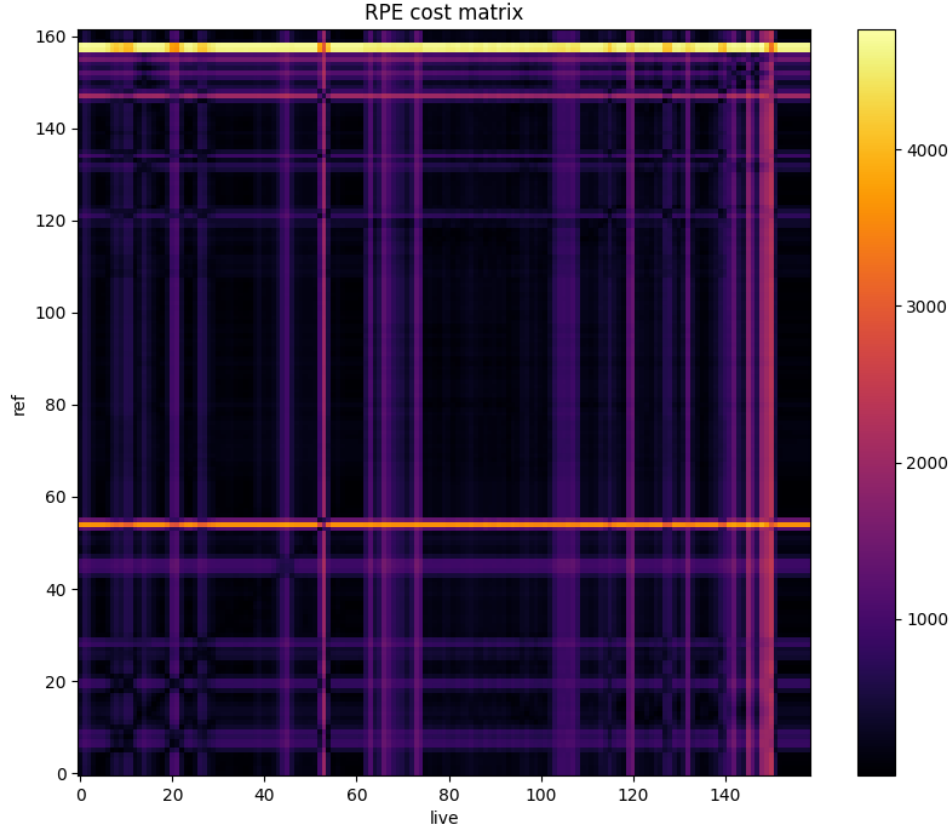


圖 3.17: 粗略位置估計器的距離成本矩陣熱力圖

當 D_{RPE} 累積現場 9 秒 (~ 30 low feature frames) 的資訊後，使用 GBA 計算時間點 t 時 Ref_{low} 所有時間點向後對齊 9 秒 ($GBA_t = 30$ frames) 的最小路徑成本矩陣 $C_{RPE} \in \mathbb{R}^{1 \times N}$ ，我們將粗略位置估計器的 GBA_{Max_Run} 設定為 3，當最小路徑成本值越小，代表此參考點與目前現場位置的相似度越高，因此我們對 C_{RPE} 做 Min-Max 資料正規化，公式如式 (3.11) 所示，所有最小成本路徑值都被正規化到 $[0, 1]$ 之間，並且相

似度高的值越接近 1。

$$S_{RPE} = 1 - \frac{C_{RPE}[n] - \min(C_{RPE})}{\max(C_{RPE}) - \min(C_{RPE})}, n = \{0, 1 \dots N\} \quad (3.11)$$

最後我們獲得相似度矩陣 $S_{RPE} \in \mathbb{R}^{1 \times N}$ ，為了取出前 8 個相似度最高的估計位置，我們對 S_{RPE} 進行降冪排序，並且設定了一個相似度閾值 θ_S ，當 S_{RPE} 中的值超過 θ_S 且排序在前 8 個順序時，才可以做為粗略估計位置輸出給決策決定者，我們參考 [51] 的設定，將為 θ_S 設為 0.95。另外累積超過 9 秒的現場音訊後，下一次的 GBA 計算會在 1.5 秒 (~ 5 low feature frames) 後，這是為了節省計算資源，使系統能保持在即時的状态。粗略位置估計器的輸出位置點如圖 3.18 的白色點所示。

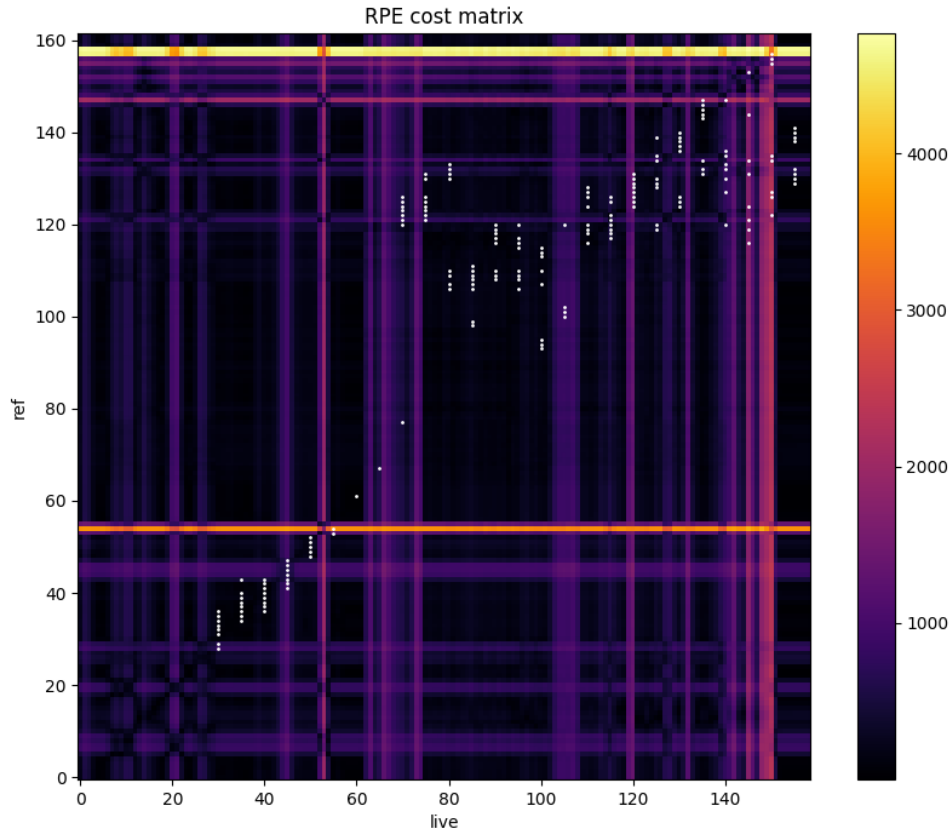


圖 3.18: 粗略位置估計器的輸出位置

3.3.7 決策決定者元件 (Decision Maker Block)

決策決定者由 ODTW thread 與 GBA threads 所組成，負責即時計算現場音訊的高解析度特徵與小提琴參考音訊的高解析度特徵之即時對齊位置，並參考粗略位置估計器的輸出位置決定最後的輸出點，為此音樂追蹤模組中最重要的區塊。

在 ODTW thread 中會持續接收現場高解析度特徵 $Live_{high}[t] \in \mathbb{R}^{1 \times F}$ ，並與參考高解析度特徵 $Ref_{high} \in \mathbb{R}^{N \times F}$ 計算兩種距離矩陣 $D_{cell} \in \mathbb{R}^{t \times N}$ 與 $D_{total} \in \mathbb{R}^{t \times N}$ ，我們定義搜尋範圍 $c = 450$ (~ 9 秒) 來限制 D_{total} 與 D_{cell} 的大小，ODTW 的計算方式與參數設定如 3.3.2 節所示，若目前沒有粗略位置估計器的輸出位置需要加入候選，則輸出 ODTW 所計算出來的對齊位置。

GBA threads 會接收來自粗略位置估計器所計算出來的輸出位置並使用 ODTW 的距離成本矩陣 D_{total} 計算 GBA 結果，為了不讓粗略位置估計器完全控制輸出結果，我們將目前 ODTW 計算出來的輸出位置 $output_j$ 與 $(output_j \pm 15, 30, 45)$ 共 7 個點一併加入 GBA threads 計算最終輸出位置。我們將 GBA_t 設為 15 秒 (~ 750 high feature frames)，每個預計輸出位置將會向後計算 15 秒的最小路徑成本並選出成本最小的位置作為最後輸出。

圖 3.19 為決策決定者輸出的熱力圖與相關元件的輸出點，綠色點為 ODTW 計算的輸出點，黃色點與桃紅色點分別為粗略位置估計器與 ODTW 計算出來的候選輸出點，GBA threads 會計算黃色點與桃紅色點的向後對齊成本，淺藍色點為 GBA 計算完後決定的輸出位置。

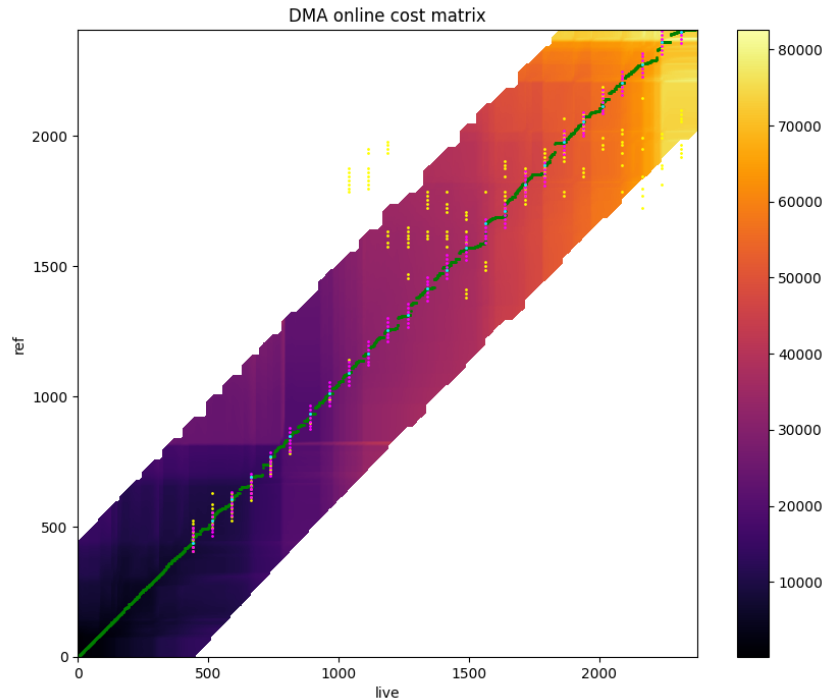


圖 3.19: 追蹤結束後決策決定者輸出的熱力圖與相關元件的輸出點

為了瞭解即時音樂追蹤系統的輸出路徑是否有偏差，我們在追蹤結束後，使用 DTW 計算現場音訊與小提琴參考音訊的輸出路徑（淺藍色點），並將系統的輸出路徑（綠色點）與 DTW 輸出路徑畫在 DTW 的累積成本矩陣熱力圖上，如圖 3.20 所示。可以看到系統輸出路徑與 DTW 的路徑在大部分的時間點都是重疊的，代表此系統在即時對齊的能力與 DTW 離線對齊的能力相當。

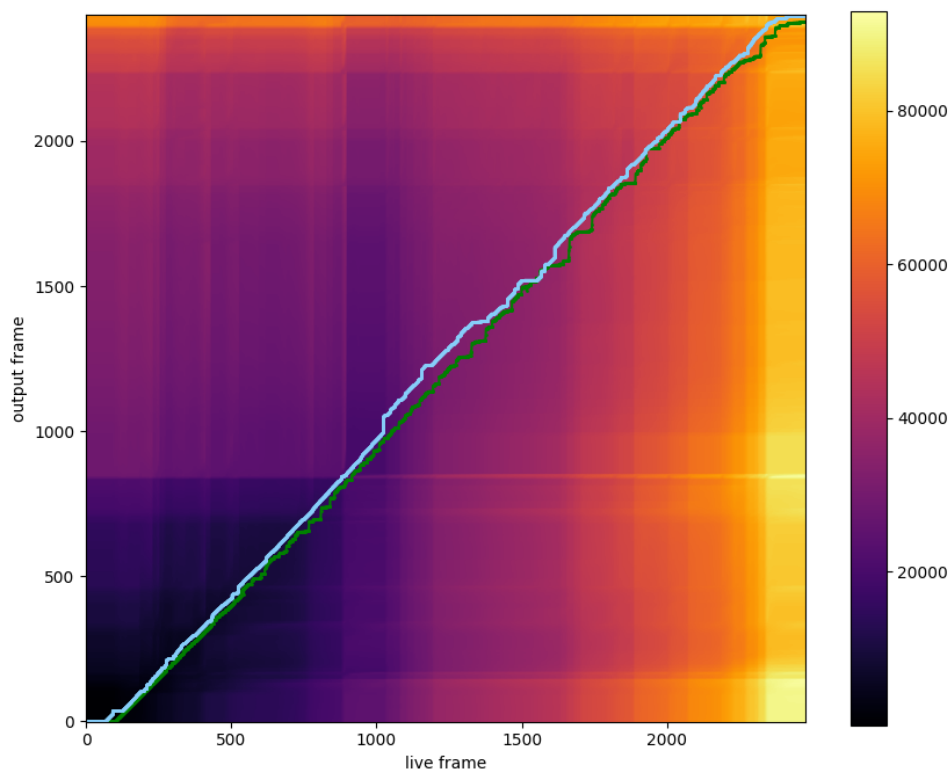


圖 3.20: DTW 與系統的輸出路徑比較圖

四、實驗設計與結果

4.1 音源分離評估

本節將比較兩個不同的音源分離模型，Aug4mss [40] 與本研究使用的 Band-Split RNN [35] 在小提琴與鋼琴混合音訊資料上的表現。首先介紹使用的訓練資料集與測試資料集，接著介紹模型訓練的設定、評估指標與評估結果。

4.1.1 音源分離資料集

在眾多的音訊公開資料集中，能同時包含多音軌錄音資料和乾淨的單音軌錄音資料可說是少之又少，在古典樂器的音訊資料集更是如此。Aug4mss 從網路上蒐集了 6 小時的古典小提琴獨奏錄音與 6 小時的流行鋼琴獨奏錄音作為他們的訓練與驗證資料集，但因為版權的關係，他們並沒有提供資料集給外界使用。因此本研究遵循 Aug4mss 蒐集資料集的方式，將小提琴與鋼琴兩種樂器的資料分開蒐集。我們整合了多個公開資料集作為訓練資料，公開資料集包含由 John Thickstun 等人提供的 MusicNet [57], [58]、Muneratti Ortega 等人提供的 Expressive Solo Violin [59] 與 Dong, Hao-Wen 等人提供的 Bach Violin Dataset [60]。

MusicNet 資料集中有 330 個古典音樂錄音，錄音種類包含鋼琴獨奏、小提琴獨奏、大提琴獨奏、長笛獨奏、鋼琴與樂器合奏、管樂合奏與弦樂合奏等組合，我們從資料集中挑選出鋼琴獨奏與小提琴獨奏的錄

音資料作為訓練資料集的一部分。Expressive Solo Violin 資料集是由專業的小提琴家在同一天錄製九個不同曲子片段的錄音，每一個片段會以不同的音樂表達方式演奏三次，並使用多個電容式麥克風同時錄音。我們採用了資料集中所有的錄音資料 (81 個片段) 作為訓練資料集的一部分。Bach Violin Dataset 為整合了高品質公開錄音的巴赫小提琴獨奏奏鳴曲 (BWV 1001-1006) 資料集，其中包含 17 位小提琴家在不同的演奏場所下錄製的資料，我們採用有提供錄音檔案的資料作為訓練資料集的一部分。

整合完畢的訓練資料集格式如表 4.1 所示，小提琴獨奏音檔的總時長為 5 小時 24 分 47 秒，鋼琴獨奏音檔的總時長為 5 小時 23 分 11 秒，所有資料皆為單聲道 WAVE 音訊格式。

表 4.1: 整合的訓練資料集格式

	小提琴音源	鋼琴音源
總時長	5hr 24min 47sec	5hr 23min 11sec
Channel	Mono	Mono
音訊格式	WAV	WAV

我們根據 [40] 中切割資料集的方式，將每種樂器的訓練資料集切成訓練集與驗證集，如表 4.2 所示。

表 4.2: 每種樂器的訓練集與驗證集大小

	小提琴音源	鋼琴音源
訓練集	4hr 22min 52sec	4hr 22min 32sec
驗證集	1hr 1min 12sec	1hr 39sec

測試資料集我們採用 Aug4mss 提供的公開整合測試資料集進行評估，此資料集是從公開資料集 MedleyDB [61] 中挑選出小提琴與鋼琴的錄音所製作的測試資料集，測試資料集的格式如表 4.3 所示。

表 4.3: 測試資料集

	混合音源
總時長	2hr 59min 35sec
Channel	Mono
音訊格式	WAV

訓練、驗證、測試資料集皆可從此連結 [62] 查看。

4.1.2 模型訓練細節

為了公平起見，我們使用自己的公開資料集重新訓練 Aug4mss 模型來評估效能，在 Aug4mss 的論文 [40] 中透過控制每次迭帶的訓練樣本數設計了 Data-limit 與 Data-rich 兩種情境，測試模型在資料缺乏與資料足夠時的表現。因此在訓練兩種模型時，我們參考 [40] 設定的資料筆數 N ， N 為在模型訓練過程中，每次迭帶從每種樂器的資料集隨機選取 N 對所混合的資料筆數，每對資料皆為 10 秒鐘的混合音訊。我們將針對上述兩種情況做測試。訓練與驗證的資料筆數 N 如表 4.4 所示，Random seed 設定為 42，確保兩個模型訓練時所拿到的資料為一致的。

表 4.4: 每次迭帶的訓練與驗證資料筆數

	Data-limit	Data-rich
訓練 資料混合筆數	$N = 250$ pairs	$N = 2000$ pairs
驗證 資料混合筆數	$N = 100$ pairs	
Random seed	42	

資料讀取與特徵擷取的參數如表 4.5 所示。

表 4.5: 資料讀取與特徵擷取參數

取樣率	44100Hz
STFT window size	2048
STFT hop size	512

兩個模型的訓練參數如表 4.6 所示，

表 4.6: 模型訓練參數

套件	PyTorch
Epoch	250
Optimizer	Adam
Learning rate	1e-3
Batch size	2
Number of workers	8
GPU	GeForce RTX™ 3080 Ti
CPU	11th Gen Intel(R) Core(TM) i7-11700K @ 3.60GHz

4.1.3 音源分離評估指標

我們使用 [35] 中的兩種評估指標來評估模型，這兩種評估指標皆是基於式 (4.1) 計算訊號失真比 (Signal-to-Noise Ratio, SDR)。

$$SDR = 10 \log_{10} \frac{\sum_n \|s(n)\|^2 + \epsilon}{\sum_n \|s(n) - \hat{s}(n)\|^2 + \epsilon}, s(n) \in \mathbb{R}^2 \quad (4.1)$$

上述公式計算同一筆資料中在離散時間 n 的訊號失真比， $s(n)$ 為正確的波型， $\hat{s}(n)$ 為估計的波型，另外為了避免除數為零，我們使用一個很小的常數 $\epsilon = 10^{-7}$ 來避免這個情況。當 SDR 的值越高，代表估計的波型與正確的波型越相近，分離效果越好。

下面介紹兩種評估指標的來源與計算方法：

1. uSDR：Utterance-level SDR 為 [24] 提出的評估方法，我們使用官方的版本針對整個訊號來計算 SDR，並計算所有曲子的平均 SDR 數值。
2. cSDR：Chunk-level SDR 為 bss_eval 指標 [63] 中計算的 SDR 方法，

我們使用官方的版本，計算每首曲子中所有 1 秒區塊的 SDR 中位數，獲得每首曲子的 SDR 數值後再取中位數。

4.1.4 音源分離結果比較

根據 4.1.2 節所提到的兩種情況，表 4.7 與表 4.8 為模型在 Data-limit 與 Data-rich 情況下訓練的結果。

另外，Aug4mss[40] 加入了多種 Data Augmentation 方法測試模型效能，由於本研究在訓練模型時皆採用相同的隨機混合方式，因此本研究只比較 [40] 中 Random mixing 的結果。

表 4.7: Data-limit 結果數值比較

分離目標樂器	模型	cSDR	uSDR
Violin	Aug4mss(paper)	1.08	N/A
	Aug4mss(retrain)	2.05	1.72
	Band-Split RNN	8.085	5.580
Piano	Aug4mss(paper)	7.43	N/A
	Aug4mss(retrain)	11.906	10.974
	Band-Split RNN	15.975	14.854

表 4.8: Data-rich 結果數值比較

分離目標樂器	模型	cSDR	uSDR
Violin	Aug4mss(paper)	3.84	N/A
	Aug4mss(retrain)	5.811	5.666
	Band-Split RNN	11.872	8.297
Piano	Aug4mss(paper)	13.46	N/A
	Aug4mss(retrain)	13.514	14.124
	Band-Split RNN	16.526	15.467

在 Data-limit 的情況下，Band-Split RNN 在小提琴和鋼琴的分離效果上都是表現最為出色。Band-Split RNN 在小提琴分離上的 cSDR 和 uSDR 分別為 8.085 和 5.580，而在鋼琴分離上的 cSDR 和 uSDR 分別為 15.975 和 14.854。相比之下，Aug4mss (retrain) 的結果分別為 2.05 和 1.72（小提琴），以及 11.906 和 10.974（鋼琴）。

在 Data-rich 的情況下，Band-Split RNN 在小提琴和鋼琴的分離效果上也是表現最為出色。Band-Split RNN 在小提琴分離上的 cSDR 和 uSDR 分別為 11.872 和 8.297，而在鋼琴分離上的 cSDR 和 uSDR 分別為 16.526 和 15.467。相比之下，Aug4mss (retrain) 的結果分別為 5.811 和 5.666（小提琴），以及 13.514 和 14.124（鋼琴）。

可以注意到 Aug4mss (retrain) 的結果皆比 Aug4mss (paper) 要來的好，可能是因為我們將原本的 STFT 參數改為與 Band-Split RNN 所設定的數值相同，原本的 Window Size 從 4096 變為 2048，Hop Size 從 1024 改為 512，因此使模型能訓練到更細部的特徵。

從表 4.7 與 表 4.8 可以看出，不管是在資料缺少或是資料足夠的情況，Band-Split RNN 皆在兩種分離目標中取得最好的數值，我認為是因為 Band-Split RNN 有特別針對重要的頻帶做細部的切割，使模型能學習到更重要的特徵。

接著我們使用上面訓練出效果最好的 BSRNN 模型來分離幾種音訊，分別為測試資料集中其中一首音訊、貝多芬第五號小提琴奏鳴曲第一樂章與韋瓦第四季-夏的第三樂章。並展示分離結果頻譜圖。

首先圖 4.1 為測試資料集中的其中一首音訊，我們展示輸入 (混合音訊)、鋼琴分離結果、鋼琴 Ground truth、小提琴分離結果與小提琴 Ground truth 的頻譜圖，可以看到第二張鋼琴分離結果與第三張鋼琴 Ground truth 頻譜圖幾乎一模一樣，而在小提琴分離結果與 Ground truth 的比較中，可以看到基頻與泛音位置幾乎沒有偏差，另外在基頻以下，Ground truth 的頻譜圖有一些錄音時會有的雜訊，但我們的分離結果中並沒有將這些雜訊也一起分離出來，我認為是因為模型並沒有針對雜訊方面特別訓練，因此剛好將雜訊濾除。

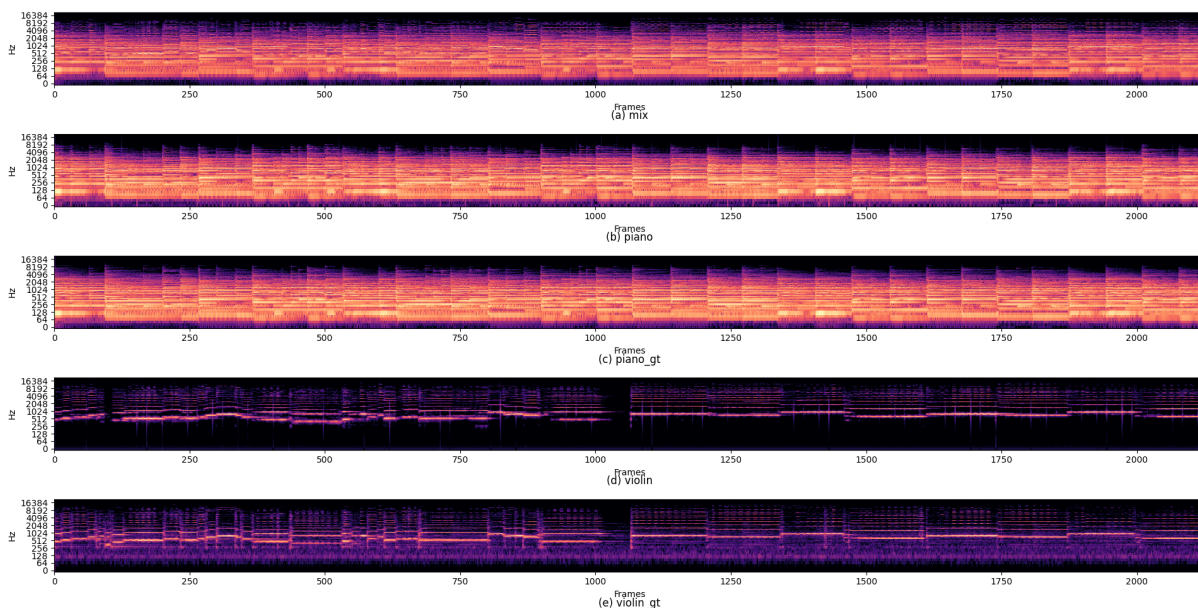


圖 4.1: 測試資料集 TheKitchenettes_Alive_p0_v0 結果比較圖：(a) 混合音訊、(b) 鋼琴音訊、(c) 鋼琴 Ground truth、(d) 小提琴音訊、(e) 小提琴 Ground truth

圖 4.2 為貝多芬第五號小提琴奏鳴曲第一樂章的結果圖，由於沒有 Ground truth 可以比較，我們只展示輸入與分離結果，這首曲子小提琴與鋼琴較沒有太多同時重疊的旋律，且旋律組成較簡單，因此可以看到分離結果都很好的將鋼琴與小提琴各自的部分分離出來。

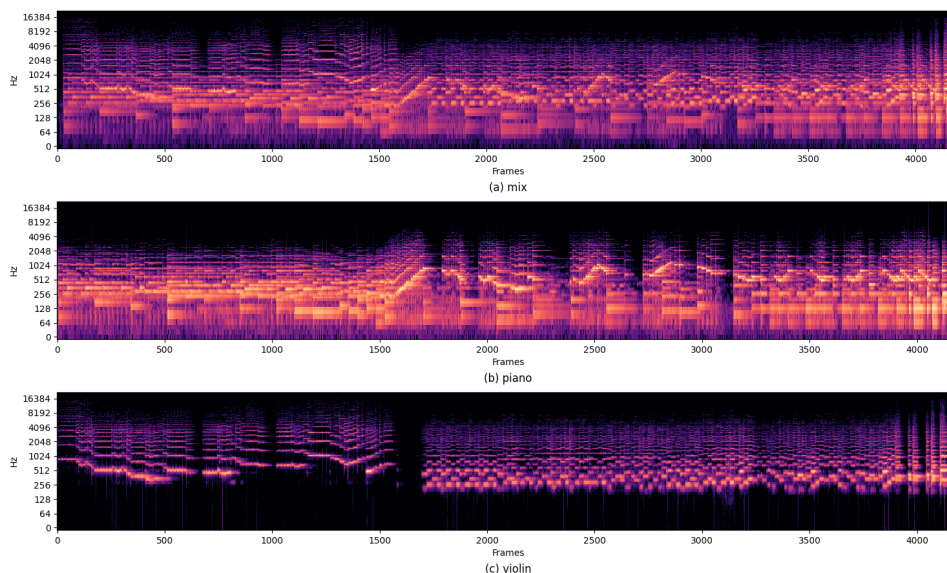


圖 4.2: 貝多芬第五號小提琴奏鳴曲第一樂章結果圖：(a) 混合音訊、(b) 鋼琴音訊、(c) 小提琴音訊

圖 4.3 為韋瓦第四季-夏第三樂章的結果圖，與上一首貝多芬相比，

這首曲子較多小提琴與鋼琴同時重疊的旋律，且速度較快，旋律組成也較複雜，因此可以看到雖然分離結果大部分都有將兩種樂器的旋律分離出來，但在相同重疊處(開頭處)可能會因為頻率成份多寡不好預測，而導致分離結果音訊聽起來可能會有一點混合音訊的聲音。

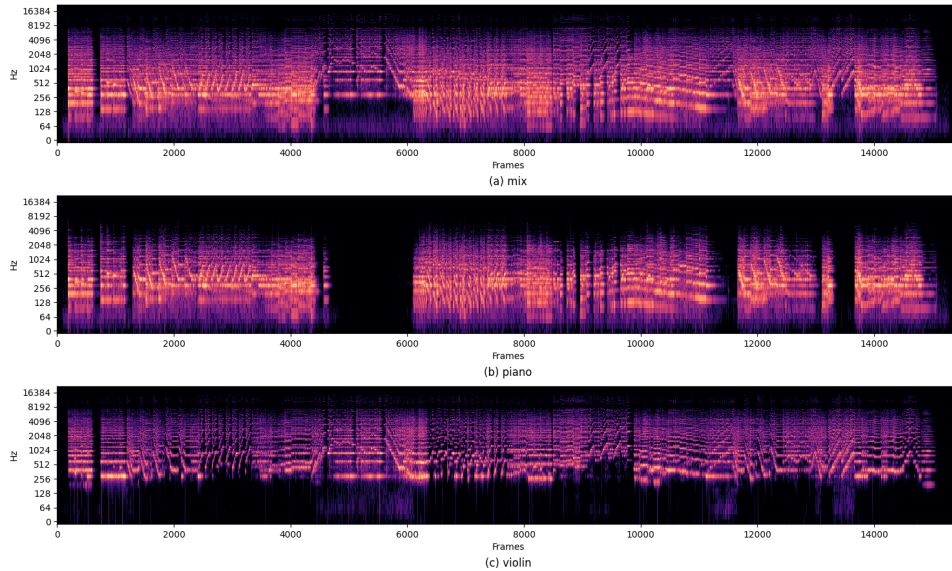


圖 4.3: 韋瓦第四季-夏 第三樂章結果圖：(a) 混合音訊、(b) 鋼琴音訊、(c) 小提琴音訊

上述所提到的三首曲子之分離結果可由此連結 [64] 查看聆聽。

4.1.5 頻帶切割對於分離結果的影響

在 BSRNN [35] 的論文中，有針對頻帶分割模組對模型結果的影響做一系列的測試，我們可以知道根據樂器不同的頻率範圍、音色特徵等等因素來調整頻帶分割的寬度，可能會提升模型的結果。

在3.2.2節我們提出了頻帶切割估計方法並針對不同的樂器設計不同的頻帶切割範圍，為了驗證此方法效果，我們針對自己設計的頻帶切割範圍，實作以下三個實驗與音源分離結果比較，

1. 使用 [35] 中效果最好的頻帶切割範圍 V7 訓練小提琴與鋼琴的音源分離模型
2. 使用小提琴的頻帶切割範圍訓練鋼琴音源分離模型

3. 使用鋼琴的頻帶切割範圍訓練小提琴音源分離模型

V7、小提琴、鋼琴的子頻帶數皆為 40 個，為了節省訓練時間，上述實驗皆使用 Data-limit 的情況訓練，我們將這些模型的結果一一列出比較，如表 4.9 所示。

表 4.9: Data-limit 不同的 Band-Split Bandwidth 結果比較

分離目標樂器	Bandwidth	cSDR	uSDR
Violin	Violin bandwidth version	8.085	5.580
	Piano bandwidth version	9.908	5.532
	V7 bandwidth version	8.842	3.157
Piano	Violin bandwidth version	15.653	14.851
	Piano bandwidth version	15.975	14.854
	V7 bandwidth version	14.556	13.370

當目標樂器為小提琴時，我們使用小提琴的頻帶切割範圍的訓練結果雖然在 cSDR 指標並不是最高，但在 uSDR 的表現上為最佳，優於使用鋼琴範圍與 V7 範圍的結果，我認為可能的原因如下，首先鋼琴音訊通常是多聲部的呈現，一首鋼琴音訊通常可獲得的頻率資訊會比一首小提琴音訊要來的多，且鋼琴與小提琴的音域有重疊的部分，小提琴的音域是包含在鋼琴音域中的，因此可能使鋼琴頻帶切割範圍估計的比小提琴還要好，導致模型訓練時獲得較好的特徵。可透過加入更多不同曲風的小提琴音訊資料增加多樣性，提升估計頻帶的效果。當目標樂器為鋼琴時，我們使用鋼琴的頻帶切割範圍的訓練結果優於使用小提琴範圍與 V7 範圍的結果，由此可見，我們所提出的頻帶分割估計方法在 uSDR 的結果上皆優於 BSRNN 提出的 V7 頻帶範圍，有助於提升音源分離的效果。

4.2 音樂追蹤評估

4.2.1 音樂追蹤模組評估方法

我們參考 [52] 中的評估方法來檢測小提琴演奏追蹤系統的表現，在 [52] 中，評估樂曲為貝多芬的 Violin Sonata No.5, Op.24(Spring), movement I. Allegro 前 25 小節，他們的現場演奏音訊為鋼琴，並且事先準備好鋼琴參考 MIDI 音訊與小提琴參考 MIDI 音訊做為音樂追蹤的參考音訊，透過即時音樂追蹤系統輸出的對齊位置即時合成估計的小提琴伴奏 MIDI 音訊。為了評估系統在不同現場演奏速度下的反應，他們使用四種不同的速度範圍來測試系統，分別為演奏正常速度 (115-145bpm)、慢速 (90-120bpm)、快速 (135-175bpm)、漸快 (由 80bpm 加速至 160bpm)，參考音訊的速度設定為 120bpm。他們錄製現場音訊與估計伴奏音訊，並使用節拍追蹤工具得到現場音訊與參考音訊之間的節拍位置，進一步生成正確伴奏音訊 (Ground truth)，最後透過 DTW 計算估計伴奏音訊與正確伴奏音訊的對齊路徑，使用式 (4.2) 與式 (4.3) 計算瞬間延遲與平均延遲來評估。

本研究的現場音訊為小提琴，估計的伴奏音訊為鋼琴，在現場音訊與參考音訊速度的設定上我們跟隨 [52] 的設定，然而在 [52] 中並沒有提到他們是如何控制現場演奏速度的變化，本研究為了能客觀地控制速度變化並計算評估結果，我們將現場音訊的部分改為使用隨機生成速度的 MIDI 檔案來評估，由於 MIDI 檔案擁有音符開始時間、節拍等等資訊，因此能精準地計算延遲，此外，我們可以透過現場 MIDI 音訊生成正確的鋼琴伴奏音訊，因此不需要額外的節拍追蹤工具輔助。雖然現場音訊為預先設定好的檔案，但這並不影響即時的效果，現場音訊還是會以串流的方式進行處理。

圖 4.4 為在四種不同速度範圍下隨機生成的 MIDI 檔案，我們透過讀取 MIDI 檔案中每個 MIDI 事件，將每個 MIDI 事件設定為有 50% 的

機率可以在速度範圍下隨機設定速度。圖 4.4a 為在慢速 (90-120bpm) 範圍的 MIDI 檔案、圖 4.4b 為在正常速度 (115-145bpm) 範圍的 MIDI 檔案、圖 4.4c 為在快速 (135-175bpm) 範圍的 MIDI 檔案，可以看到每個有設定速度的 MIDI 事件，其速度設定都在對應的範圍中。另外圖 4.4d 為漸快 (由 80bpm 加速至 160bpm) 範圍的 MIDI 檔案，我們平均的將速度範圍分配到每個 MIDI 事件上，模擬漸快的情況。

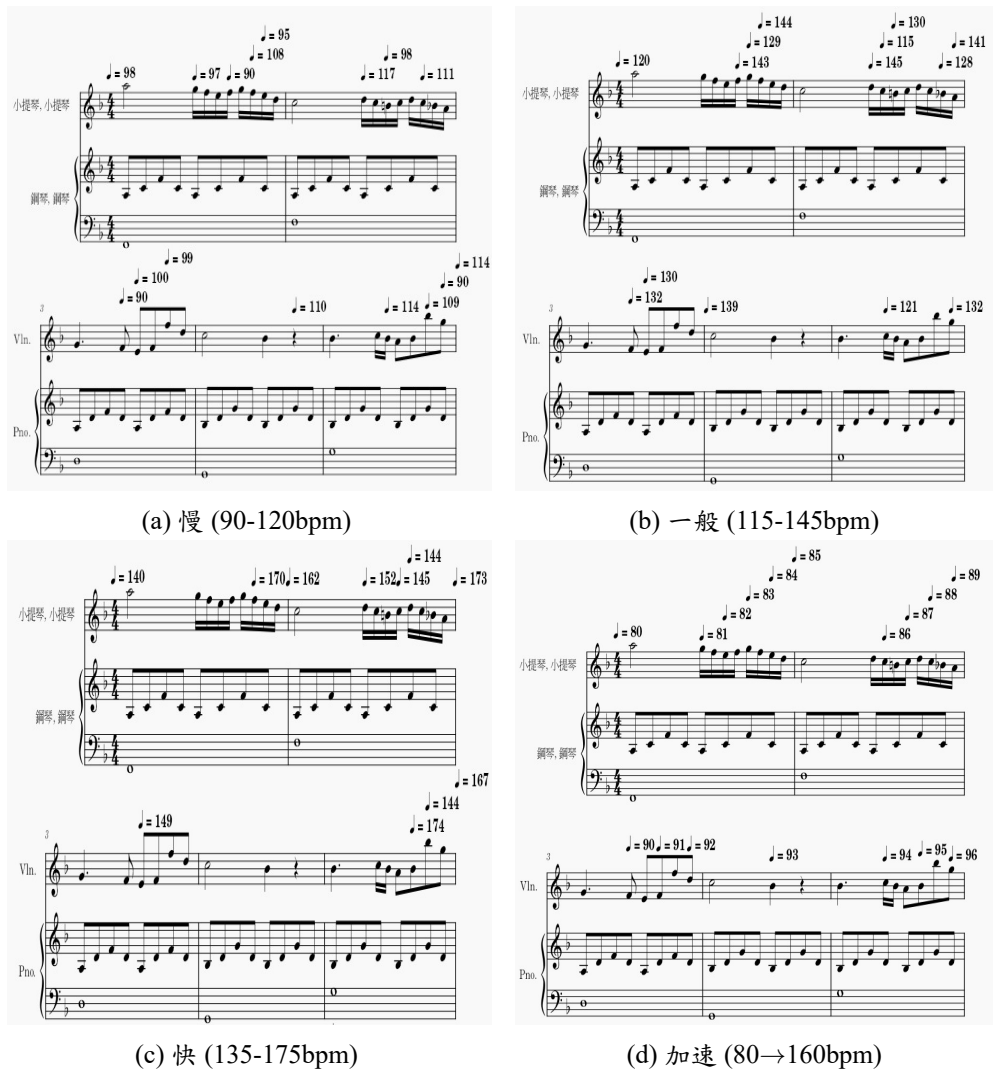


圖 4.4: 四種不同速度範圍的 MIDI 樂譜前五小節

由於正確的鋼琴伴奏音訊已經透過 MIDI 檔案生成，我們只需要用正確伴奏音訊 Acc_{gt} 與即時音樂追蹤系統估計的伴奏音訊 Acc_{out} 計算 DTW 對齊路徑，由於伴奏音訊的時間長度是一致的，因此可以根據

式 (4.2) 算出 Acc_{out} 在 Acc_{gt} 上的瞬間延遲 (latency)。

$$\Delta[t_i] := t_j - t_i \quad (4.2)$$

t_i 為 Acc_{out} 的時間點， t_j 為 Acc_{gt} 的時間點，當 $t_i = t_j$ 時，代表在這個時間點兩段音訊是對齊的，當 $t_i > t_j$ 時，代表 Acc_{out} 比 Acc_{gt} 還要快，當 $t_i < t_j$ 時，代表 Acc_{out} 比 Acc_{gt} 還要慢。

我們也用式 (4.3) 衡量整首曲子的平均延遲，計算方式為將每個時間點的瞬間延遲取絕對值後加總，並除以曲子的長度 T 。 $|\Delta t|$ 為估計伴奏音訊的延遲。

$$\frac{1}{T} \sum_{t=0}^T |\Delta[t]| \quad (4.3)$$

4.2.2 音樂追蹤模組評估結果

我們顯示系統在四種速度變化下隨時間變化的延遲圖，也就是估計伴奏音訊比正確伴奏音訊要提前或落後多少個 16 分音符。藍色線為系統延遲的 16 分音符數量，橘色線為當下的速度 (bpm)。圖 4.5 是慢速時系統隨時間變化的延遲，可以看到系統在一開始雖然動盪較大，但都保持在不超過 2 個 16 分音符的延遲下穩定追蹤，即使節拍動盪很大也不影響到系統的穩定性，最後的平均延遲為 55.04ms。

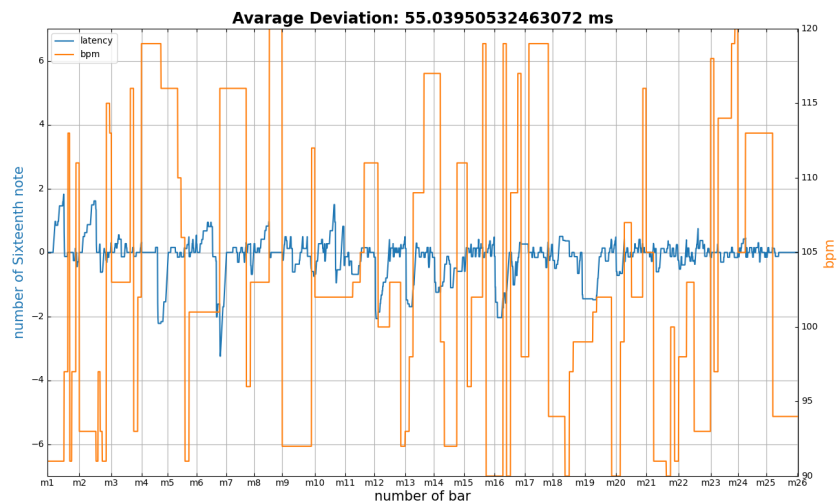


圖 4.5: 慢 (90-120bpm) 平均延遲：55.04 毫秒

圖 4.6 是一般速度下的延遲圖，由於一般速度的範圍是最接近參考音訊的速度，因此可以看到整體平均延遲為四種速度下最低的，系統延遲線幾乎保持在 y 軸為 0 的地方。

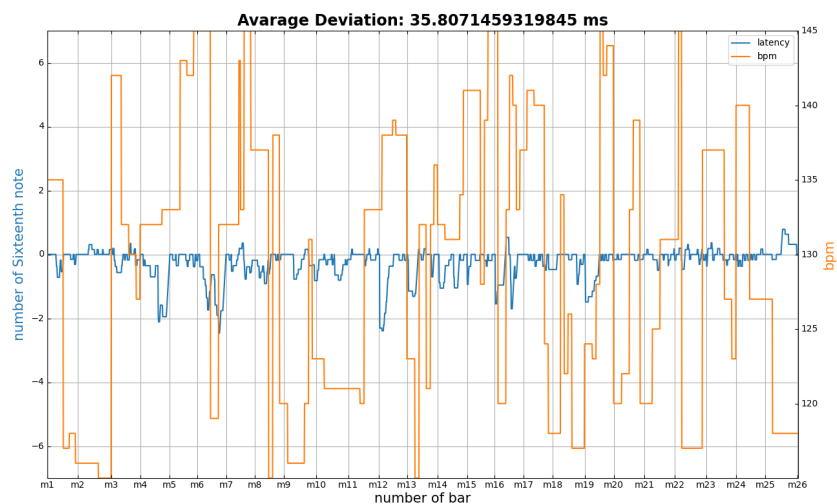


圖 4.6: 一般 (115-145bpm) 平均延遲：35.81 毫秒

圖 4.7 是快速時系統隨時間變化的延遲，與慢速的延遲圖很像，但動盪較大，這可能是因為快速的速度範圍與參考的速度相差較多，因此導致系統在追蹤時較容易有延遲，最後的平均延遲為 62.96ms。

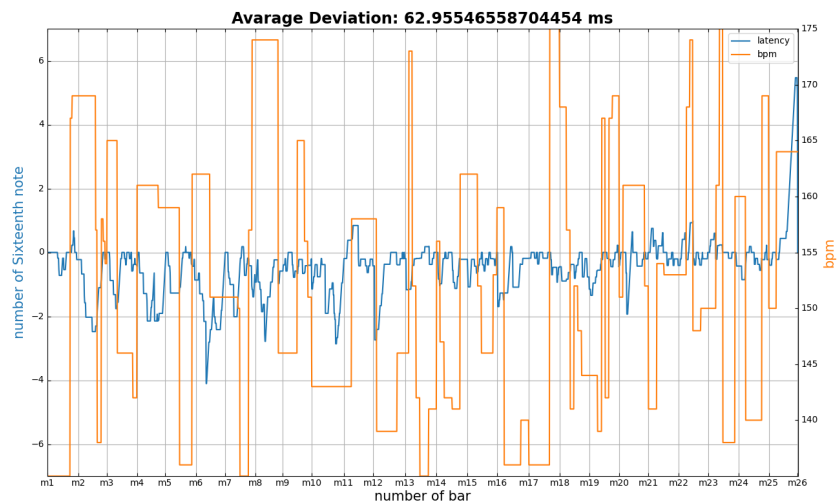


圖 4.7: 快 (135-175bpm) 平均延遲：62.96 毫秒

圖 4.8是漸快時系統隨時間變化的延遲，由於一開始速度較慢，因此導致系統剛開始追蹤時動盪較大，當速度慢慢回升到接近參考音訊的速度時，可以看到動盪越來越小，最後的平均延遲為 58.23ms。

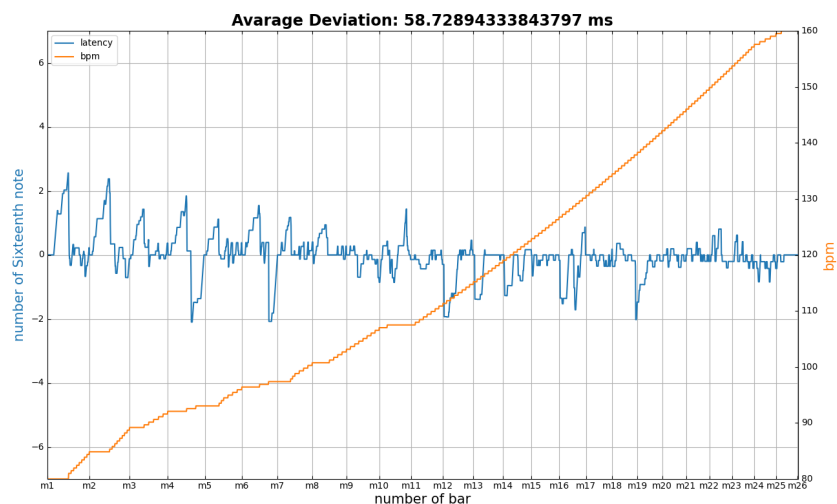


圖 4.8: 加速 (80→160bpm) 平均延遲：58.73 毫秒

另外可以發現在第五小節 (m5) 與第七小節 (m7) 時，四種速度的 16 分音符延遲數皆為負數，也就是比起正確的時間點還要提前，可能是因為剛好那個時間點為休止符，也就是沒有音樂，因此造成系統先對齊到了最後面的靜音時間點並等待，導致延遲。

我們也顯示了四種速度的對齊路徑結果圖，橫軸為現場音訊的時間點，縱軸為小提琴參考音訊的時間點，熱點圖的顏色代表在時間點 (i, j) 的累積距離成本值，成本值越高顏色越亮。(a) 圖為追蹤系統在執行時所輸出的結果圖，(b) 圖為離線對齊路徑與線上對齊路徑的比較圖，離線對齊路徑使用 DTW 計算。

圖 4.9 在大部分的時間點，速度會比參考音訊的速度要慢，因此可以看到圖 4.9a 在某些區間 (例如 live 約 600frames 時) 綠色的點所連成的線較為平緩，代表此區間系統輸出較多相同的參考音訊時間點來等待現場音訊。在圖 4.9b 的表現上，幾乎與離線對齊時的最佳路徑是一致的。

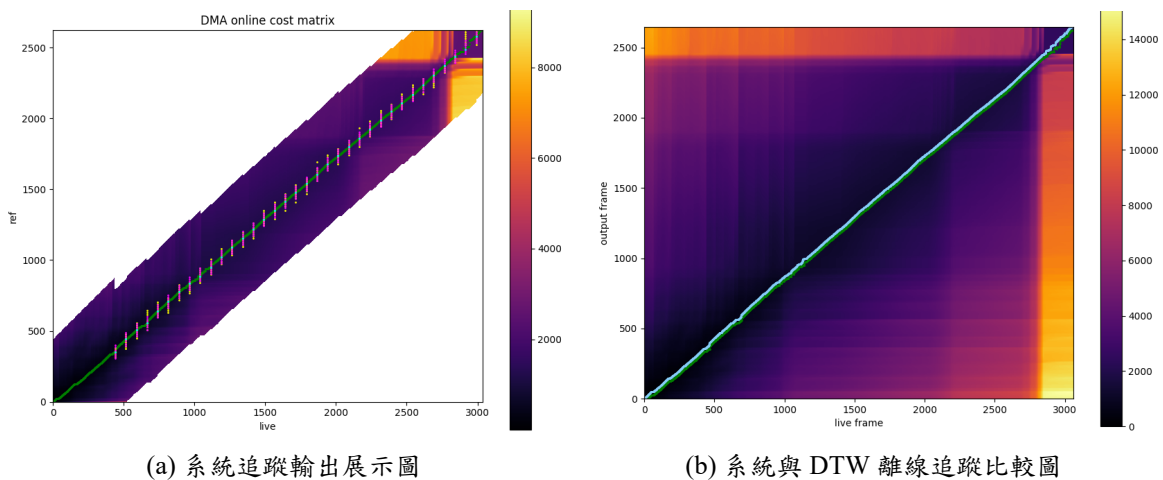


圖 4.9: 慢 (90-120bpm)

圖 4.10 的速度與參考音訊是最接近的，因此在圖 4.10a 幾乎不會有波動，圖 4.10b 也是表現得與離線對齊路徑相當。

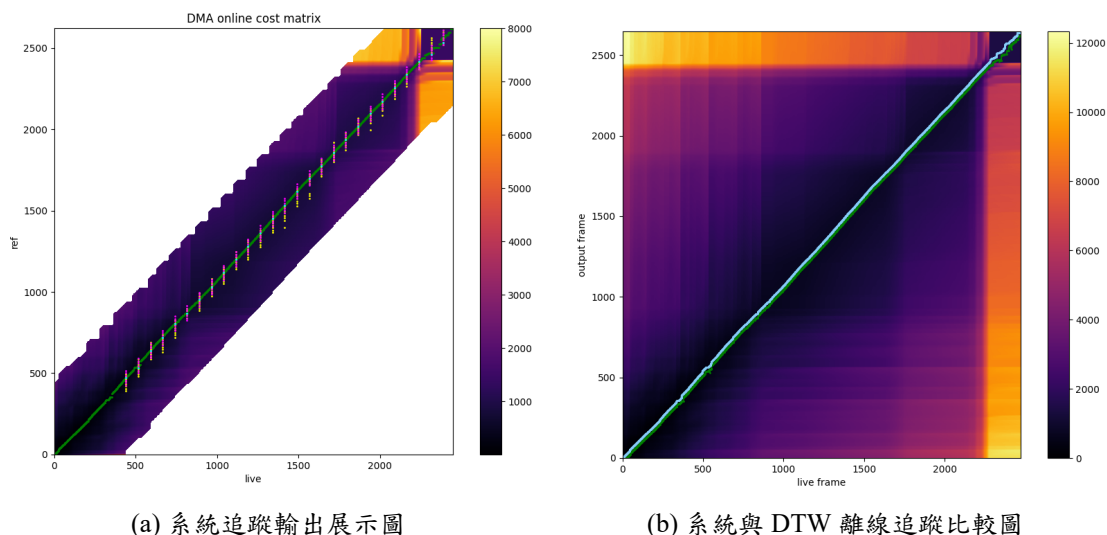


圖 4.10: 一般 (115-145bpm)

圖 4.11 在大部分的時間點，速度會比參考音訊的速度要快，因此可以看到圖 4.11a 在某些區間 (例如 live 約 450frames 時) 輸出路徑會呈現斷斷續續的點，代表此區間系統在輸出時認為目前現場音訊的時間點必須對齊到更後面的參考音訊，在圖 4.11b 的表現上，雖然參考音訊的時間點並不是連續的，但可以看到整體路徑與離線對齊路徑還是一致的。

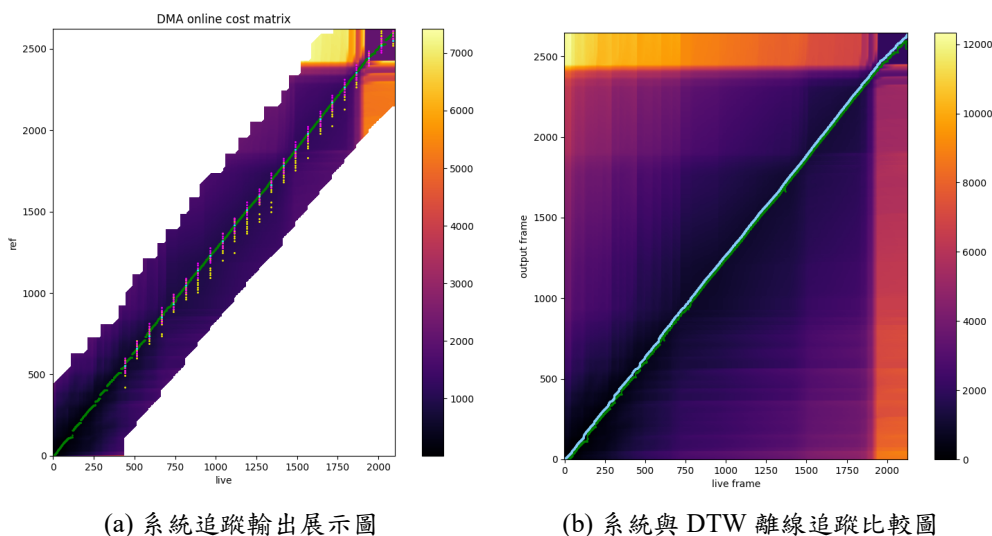
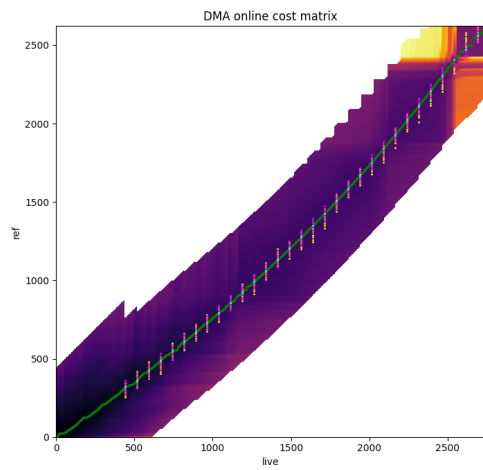


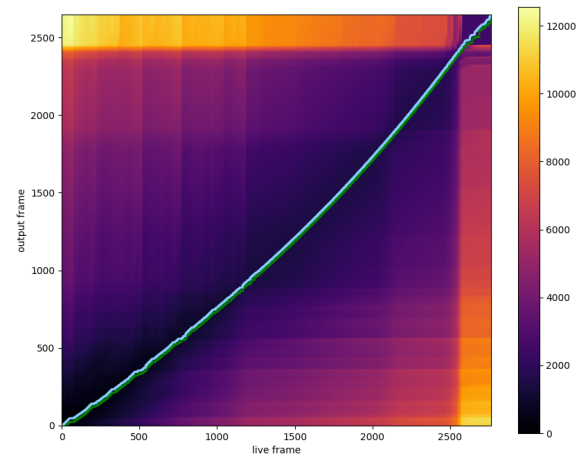
圖 4.11: 快 (135-175bpm)

圖 4.12 涵蓋了整個慢到快的速度區間，可以看到圖 4.12a 在前面速度較慢時輸出相同的點的狀況較多，當現場音訊演奏越來越快，輸出點所連成的線的斜率越來越高。在圖 4.12b 的表現上，與離線對齊路徑也是呈

現一致。



(a) 系統追蹤輸出展示圖



(b) 系統與 DTW 離線追蹤比較圖

圖 4.12: 加速 (80→160bpm)

上述使用的 MIDI 檔案與結果音檔可從此連結 [65] 查看聆聽。

4.3 整體系統評估

4.3.1 系統評估方法

為了評估系統是否在不同特徵下也能有好的追蹤結果，我們使用與4.2.1節相同的曲子，在網路上找到由 kopikostar 演奏的小提琴與鋼琴合奏版本 [66]，並對此混合音訊做音源分離，將分離結果作為音樂追蹤模組的參考音訊。分離結果的音檔可至此連結聆聽 [67]。

我們實際演奏現場音訊，並演奏了四種不同速度，分別為 110bpm、120bpm、130bpm 與 140bpm，此四種速度為這首曲子最常出現的演奏速度。演奏時，為了保持演奏速度的平穩，我們皆使用節拍器來輔助，我們也演奏了自由速度版本，也就是不使用節拍器，與鋼琴互相配合演奏。在演奏時使用 AKG C519 M 電容式麥克風收錄音訊，監聽耳機為 Sony MDR 7506，並使用 Focusrite Scarlett 2i2 錄音介面處理類比與數位訊號，硬體設備如圖 4.13所示。

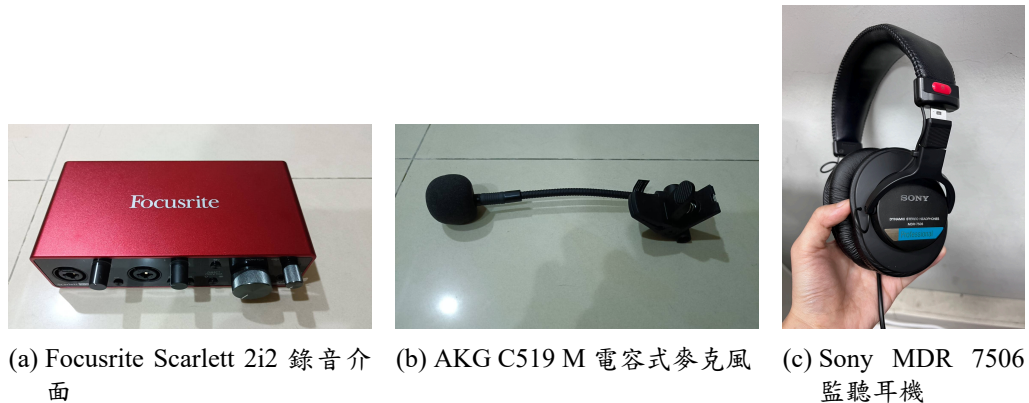


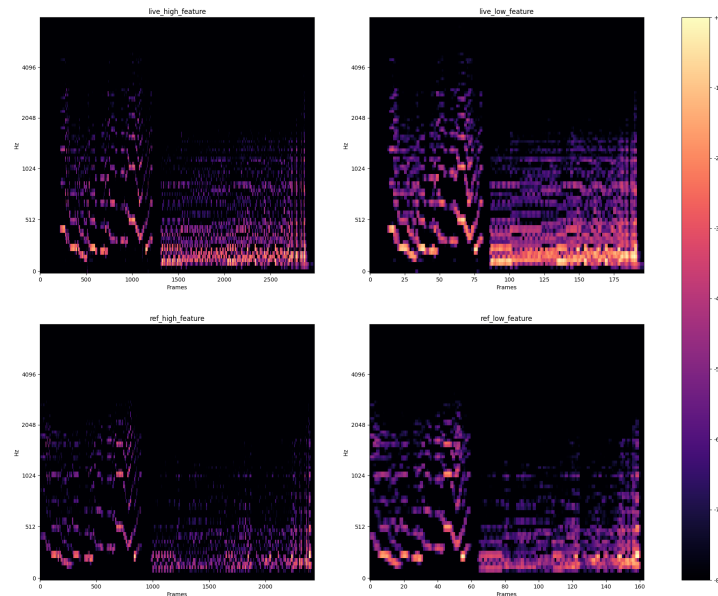
圖 4.13: 硬體設備

由於沒有正確的伴奏音訊 (Ground truth)，因此我們無法使用4.2.1節的方法來評估與正確伴奏音訊的延遲。因此我們改由計算系統輸出路徑與 DTW 離線對齊路徑的平均延遲幀數來評估系統在線上追蹤的表現是否與離線追蹤相當，這邊的平均延遲幀數代表的是對於線上追蹤表現，是否提前或落後離線追蹤表現。

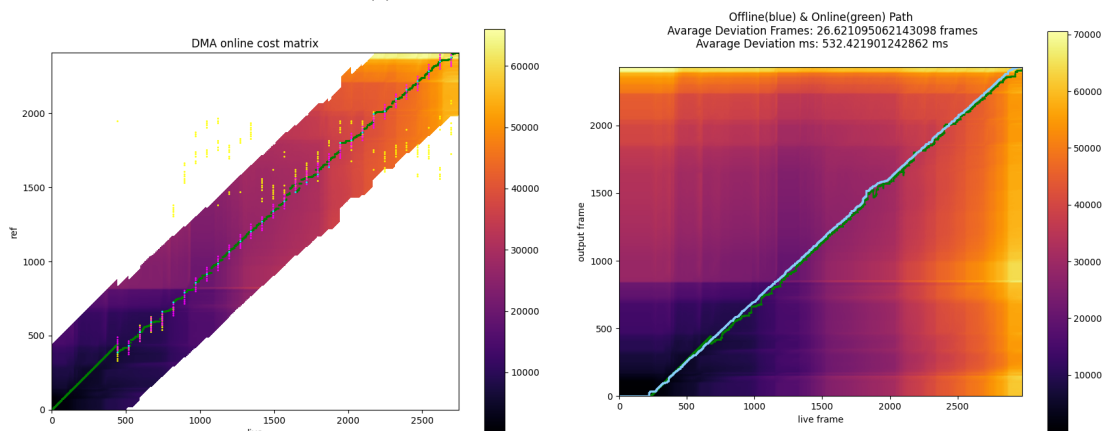
4.3.2 系統評估結果

下圖為五種不同速度的對齊路徑結果圖，橫軸為現場音訊的時間點，縱軸為小提琴參考音訊的時間點，(a) 圖為追蹤系統在執行時所輸出的結果圖，(b) 圖為離線對齊路徑與線上對齊路徑的比較圖與平均延遲幀數。

圖 4.14 為速度 110bpm 下的追蹤結果，由圖 4.14b 的 xy 軸可以看出現場音訊的速度比小提琴參考音訊要來的慢，因此會出現等待的情況，例如 live frame=700 的時候。圖 4.14c 顯示了追蹤結果與離線計算結果的延遲為 26.62 幀，約為 532ms 左右。



(a) 現場音訊與參考音訊之特徵圖



(b) 系統追蹤輸出展示圖

(c) 系統與 DTW 離線追蹤比較圖

圖 4.14: 110bpm 平均延遲：26.62 幀

圖 4.15 為速度 120bpm 下的追蹤結果，由圖 4.15b 的 xy 軸可以看出現場音訊的速度也是比小提琴參考音訊的速度稍慢，圖 4.15c 顯示了追蹤結果與離線計算結果的延遲為 27.98 幀，約為 560ms 左右。

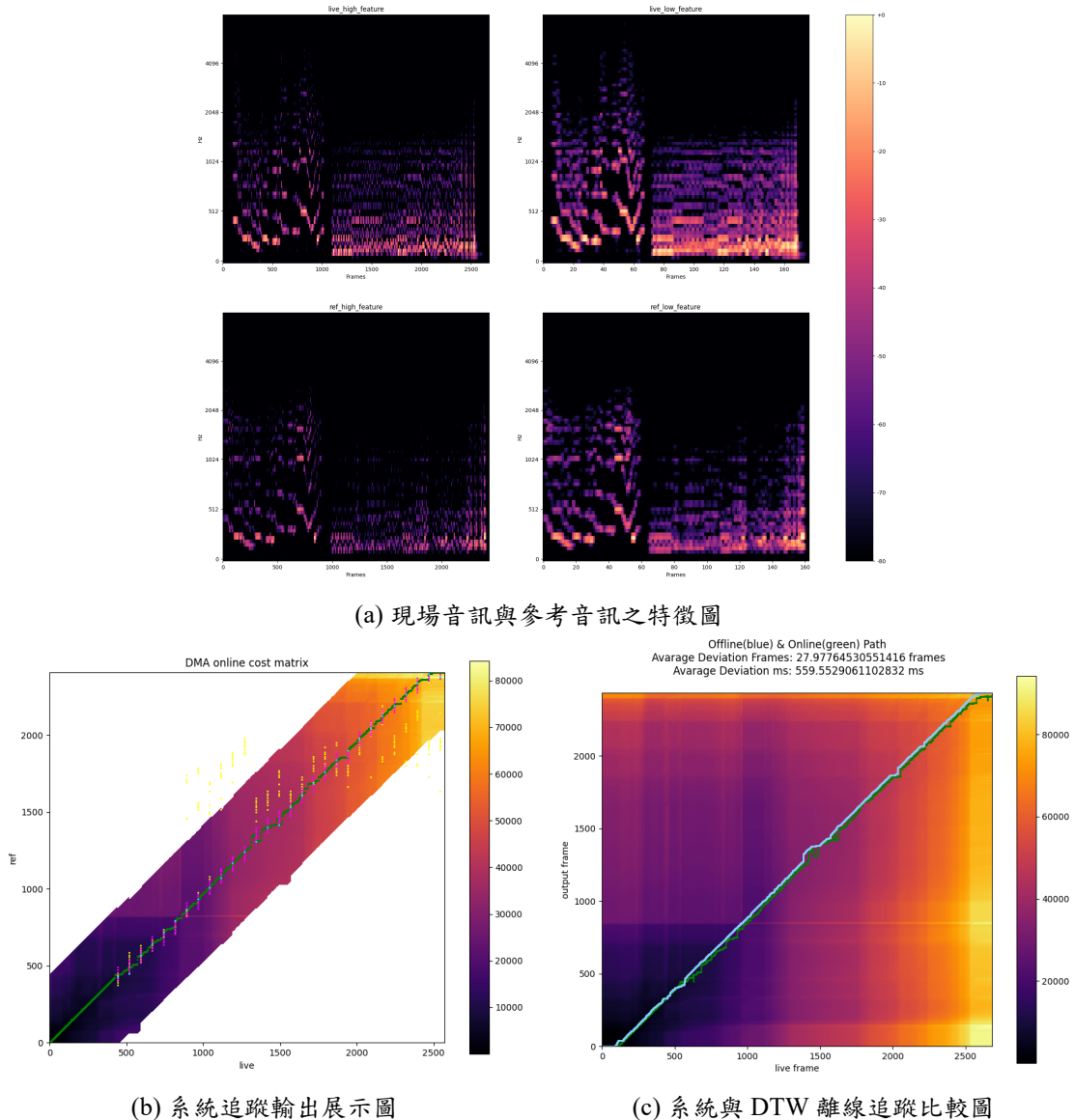
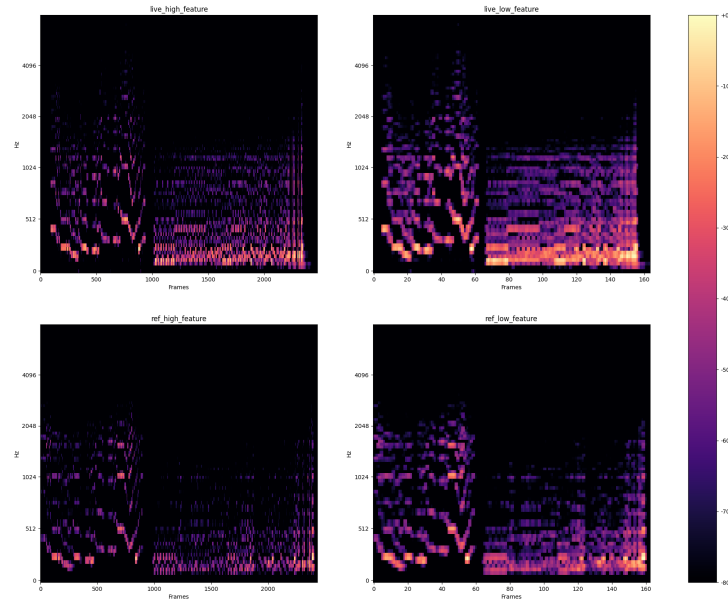
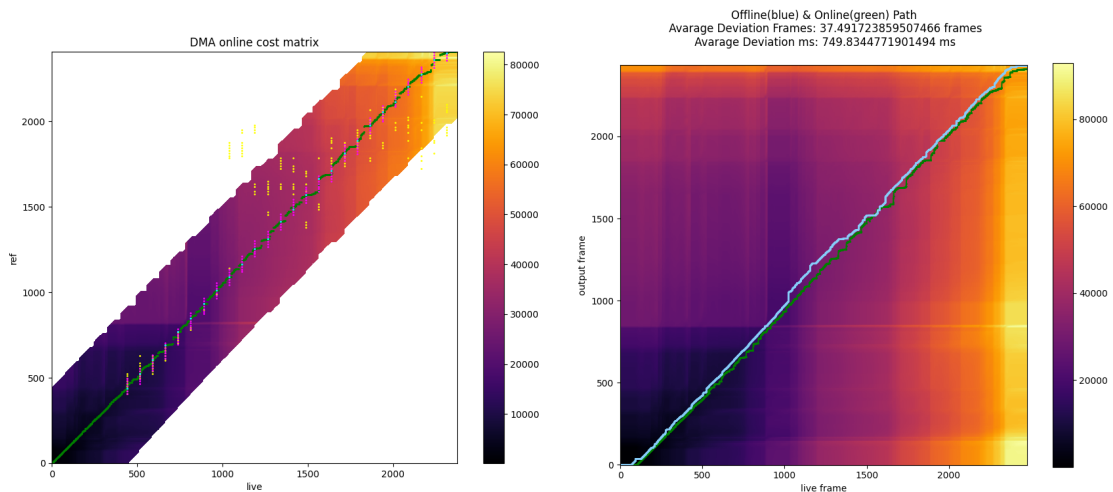


圖 4.15: 120bpm 平均延遲：27.98 幀

圖 4.16 為速度 130bpm 下的追蹤結果，由圖 4.16b 的 xy 軸可以看出現場音訊與參考音訊的速度較為相近，但可能因為特徵差距的關係使追蹤結果並沒有比起前面的結果要好。圖 4.16c 顯示了追蹤結果與離線計算結果的延遲為 37.49 幀，約為 750ms 左右。



(a) 現場音訊與參考音訊之特徵圖

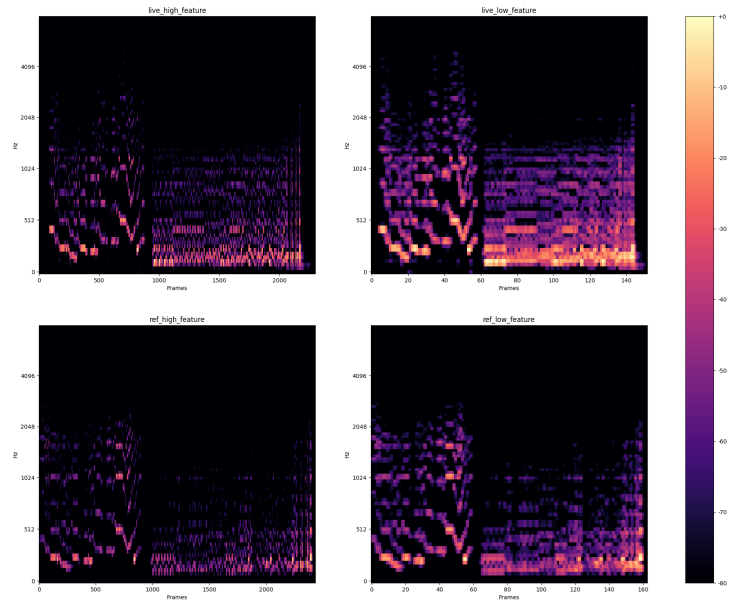


(b) 系統追蹤輸出展示圖

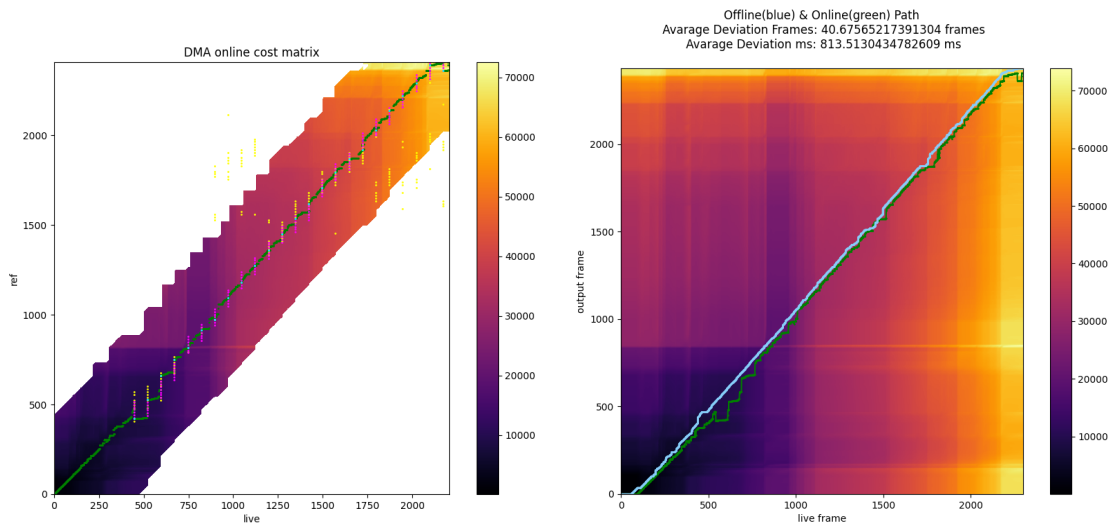
(c) 系統與 DTW 離線追蹤比較圖

圖 4.16: 130bpm 平均延遲：37.49 幀

圖 4.17 為速度 140bpm 下的追蹤結果，由圖 4.17b 的 xy 軸可以看出現場音訊比小提琴參考音訊的速度稍快一些，在 live frame 約 500 的時候，可以看到追蹤的路徑並不穩定，此時靠著 RPE 的粗略估計位置將 ODTW 對齊錯誤的地方拉回正軌。圖 4.17c 顯示了追蹤結果與離線計算結果的延遲為 40.68 幀，約為 814ms 左右。



(a) 現場音訊與參考音訊之特徵圖

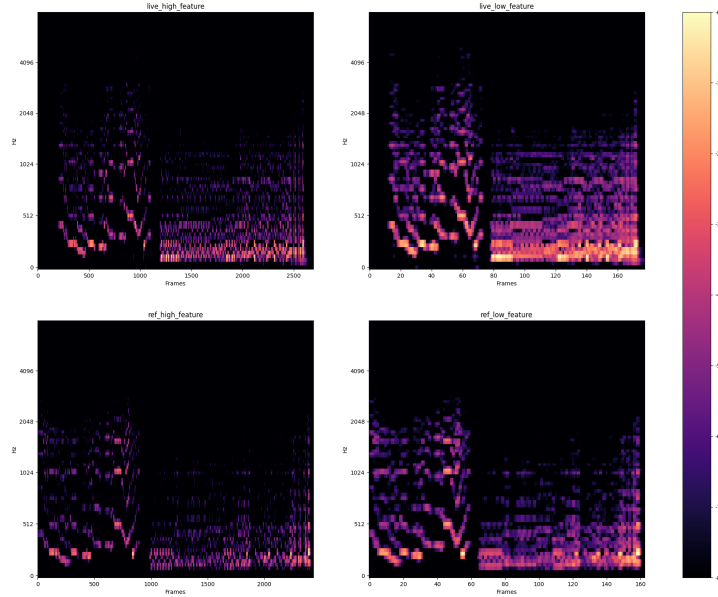


(b) 系統追蹤輸出展示圖

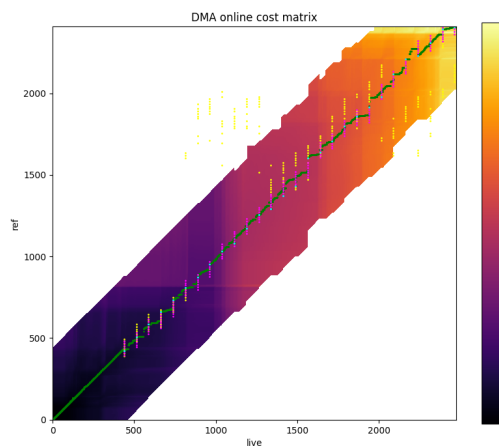
(c) 系統與 DTW 離線追蹤比較圖

圖 4.17: 140bpm 平均延遲：40.68 幀

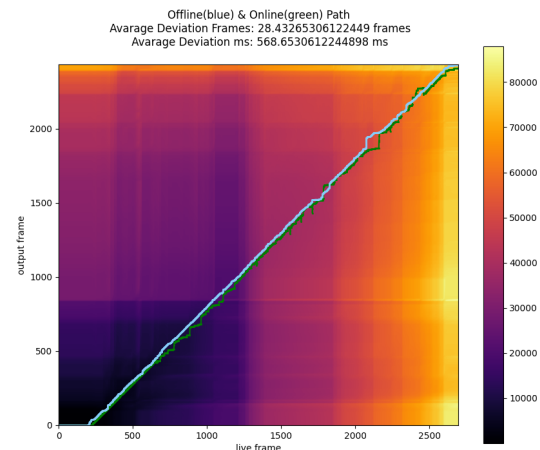
圖 4.18 為自由速度下的追蹤結果，由圖 4.18b 的 xy 軸可以看出現場音訊比小提琴參考音訊的速度稍慢一些，在 live frame 約 1500 之後，可以看到追蹤的路徑並不穩定，我認為可能是這段的旋律是重複的，因此可能在粗略估計的位置上無法精準的計算目前粗估位置，但最後還是有走回正軌。圖 4.18c 顯示了追蹤結果與離線計算結果的延遲為 28.43 幀，約為 569ms 左右。



(a) 現場音訊與參考音訊之特徵圖



(b) 系統追蹤輸出展示圖



(c) 系統與 DTW 離線追蹤比較圖

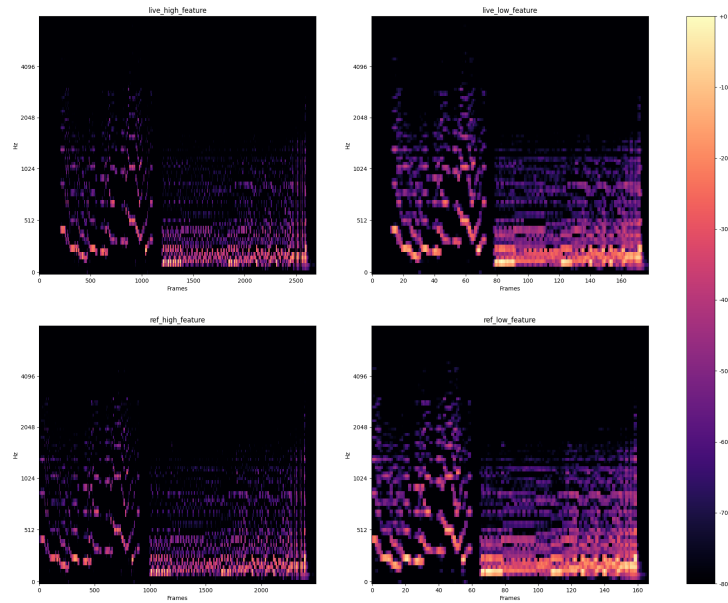
圖 4.18: 自由速度平均延遲：28.43 幀

由於參考音訊與現場音訊的特徵不一致，導致 ODTW 在計算對齊點時穩定性較為不足，RPE 估計出的位置也因為兩個音訊的特徵不夠相近，使粗略估計的位置偏差較嚴重，因此追蹤結果圖大多數看起來比4.2.2節的結果要差。但可以看到在比較離線對齊路徑與線上對齊路徑的結果圖，大致上的路徑都與離線對齊路徑重合，因此我們可以知道這兩個音訊在線上的最佳對齊路徑已經非常接近離線對齊路徑，達到了即使使用不同特徵追蹤也能維持離線最佳對齊的效果。

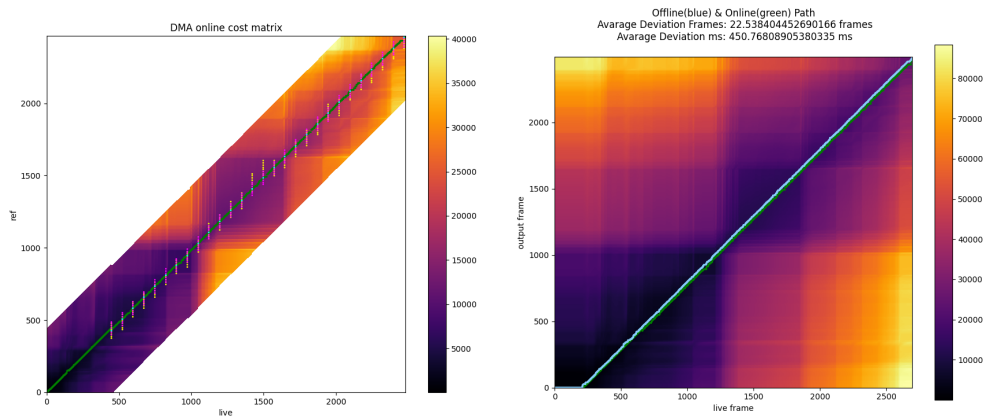
另外可以看到比較圖中從第 0 幀到兩段路徑開始對齊的時間點是一致的，代表我們的音樂偵測元件在判斷演奏者的演奏時間是精準的。此

外，我認為平均延遲都高達 500ms 以上可能是因為 DTW 在計算對齊路徑時， i 與 j 都有遵守連續性的限制，因此不會出現一次跳 2 格以上的路徑，且 i 是可以重複的；但我們在計算 ODTW 時，在 j 方向是有可能出現一次跳 2 格以上的路徑，然而 i 為嚴格遞增，因此造成兩個路徑在計算時，因為固定了 i 的位置，在 $(t_j - t_i)$ 的差距可能會更大，但實際聽起來並沒有延遲 500ms 那麼多。

我們也測試了使用同一人演奏的參考特徵來追蹤現場演奏的效果，首先我們先錄製了與伴奏音訊完整對齊的音訊作為混合音訊，並將此混合音訊透過音源分離模型分離成參考音訊，並使用這些音訊進行音樂追蹤。接著現場演奏我們一樣採自由速度，聆聽伴奏音訊來演奏，結果如圖 4.19 所示，可以看到線上對齊路徑與離線對齊路徑重合，代表系統輸出的路徑是最佳對齊位置，在平均延遲上也比圖 4.18 要來的好。



(a) 現場音訊與參考音訊之特徵圖



(b) 系統追蹤輸出展示圖

(c) 系統與 DTW 離線追蹤比較圖

圖 4.19: 自由速度 (參考音訊為同一人所演奏) 平均延遲：22.54 幀

上述的結果音檔可從此連結 [68] 查看聆聽。

最後，根據 [69] 的文章寫到，[52] 的系統在偵測到觸發伴奏的平均延遲控制在 0.1 秒左右，而本研究實際測試後，我們的系統可將偵測到觸發伴奏 (接收串流音訊到輸出伴奏音訊所花費的時間) 的平均延遲控制在 0.05 秒左右。

五、 總結

5.1 結論

本研究提出了一套結合音源分離模組與音樂追蹤模組的小提琴演奏追蹤系統，此系統能將小提琴與鋼琴的混合音源分離成參考音源，並結合現場小提琴音訊實現即時追蹤與伴奏的效果。在音源分離模組方面，本研究採用 Band-Split RNN 模型並提出了簡單的頻帶切割方法，有效提升模型的分離效果。在音樂追蹤模組方面，本研究改進了音訊特徵擷取方式，即時處理串流音訊使其他模塊能使用音訊特徵。此外，我們也改良了音樂偵測模塊，加入了平均振幅閾值判斷法加速過濾靜音片段。我們也改進了線上動態時間規整演算法 (ODTW)，使系統能夠更精確地對齊現場音訊並即時輸出。最後，改進了貪心向後對齊演算法 (GBA)，使模組在決定輸出位置時能參考更精準的資訊。

研究實驗結果顯示，音源分離模型在資料缺乏與資料充足的情況下，效果皆優於現有的基線模型。針對加入頻帶切割方法是否對模型有正面影響的實驗結果表明，對不同樂器使用頻帶切割方法可提升模型效果。另外，音樂追蹤模組的實驗結果顯示，在隨機速度下的即時追蹤效果最好的延遲為 35.81ms，最差的結果也只延遲了 4 個 16 分音符，即使從慢版速度 (80bpm) 加速到快板速度 (160bpm)，即時追蹤效果也同樣穩定。整體系統的即時追蹤效果，雖然比使用相同特徵追蹤的效果要差，但與離線追蹤效果相比，系統的即時追蹤最佳路徑幾乎與離線追蹤的最佳路徑重疊，顯示即時追蹤效果也能達到與離線追蹤相同的穩定度與精準度。

5.2 未來展望

本研究成功在使用不同特徵且即時的情況下追蹤小提琴演奏的位置並輸出伴奏音訊，但此系統還有許多可以改進的地方，未來的研究可以探索以下幾個方向。

在音源分離方面，本研究使用了較輕量的 BSRNN 模型進行訓練，在頻帶切割估計方法中，使用人為調整參數的方式切割頻帶，未來可嘗試使用簡單的非監督式學習演算法 (例如：K-means) 來預估頻帶切割點。另外，未來可以嘗試使用更大的模型，例如目前效果最佳的模型 Hybrid Transformers [38]。此模型能同時在時頻域和波形域進行訓練，且不需要訓練多個分離目標模型。此外，本研究只針對小提琴與鋼琴的混合音源進行研究，但其實在古典音樂中，還有許多不同的樂器合奏配置，且 BSRNN 為靈活度高的模型，透過更改訓練資料集的資料，即可訓練不同目標的音源分離模型，因此，未來希望能蒐集更多不同樂器的訓練資料，擴展音源分離技術以涵蓋更多種類的古典樂器。

在音樂追蹤方面，可以加入偵測音高的深度學習模型，使追蹤時可以獲得參考音訊與現場音訊的音高資訊，讓追蹤系統同時獲得音高與音色的資訊，從而更精準的預測目前演奏位置。另外本系統的輸出位置若連續停留在同一位置，會導致伴奏聽起來有失真或雜訊的感覺，未來可以研究輸出音訊的平滑化技術，或是改由使用輸出位置估計目前的演奏速度來調整整段伴奏的快慢。另外，可以結合數位音訊工作站 (Digital Audio Workstation, DAW)，將輸入與輸出皆透過 DAW 處理，使程式更簡化，並且能更快速地轉換特徵，從而降低系統延遲。

希望未來能在這些方面進行深入研究，以進一步提升系統的性能和應用範圍。

參考文獻

- [1] C.A.P.E. “Statistics on the number of applicants for music subjects over the years.” (2024), [Online]. Available: <https://www.cape.edu.tw/statistics/> (visited on 05/07/2024).
- [2] 我是江老師. “鋼琴伴奏月薪？一小時賺多少？到底都在做什麼？” (2020), [Online]. Available: <https://youtu.be/8MBaTBXLzEw?t=100> (visited on 05/07/2024).
- [3] A. Défossez, N. Usunier, L. Bottou, and F. Bach, *Music source separation in the waveform domain*, 2021.
- [4] E. Cano, D. FitzGerald, A. Liutkus, M. D. Plumbley, and F.-R. Stöter, “Musical source separation: An introduction,” *IEEE Signal Processing Magazine*, vol. 36, no. 1, pp. 31–40, 2019.
- [5] Z. Rafii, A. Liutkus, F.-R. Stöter, S. I. Mimilakis, D. FitzGerald, and B. Pardo, “An overview of lead and accompaniment separation in music,” *IEEE/ACM Transactions on Audio, Speech, and Language Processing*, vol. 26, no. 8, pp. 1307–1335, 2018.
- [6] M. E. P. Davies, “Towards automatic rhythmic accompaniment,” Ph.D. dissertation, Citeseer, 2007.
- [7] Y. Li, “Application of computer-based auto accompaniment in music education,” *International Journal of Emerging Technologies in Learning (iJET)*, vol. 15, no. 6, pp. 140–151, 2020.
- [8] X. Zhang and C. Liu, “Design of piano automatic accompaniment system based on artificial intelligence algorithm,” in *International Conference on Computational Finance and Business Analytics*, Springer, 2023, pp. 249–258.
- [9] N. Orio, S. Lemouton, and D. Schwarz, “Score following: State of the art and new developments,” *New Interfaces for Musical Expression (NIME)*, 2003.
- [10] M. Dorfer, A. Arzt, and G. Widmer, “Towards score following in sheet music images,” *arXiv preprint arXiv:1612.05050*, 2016.
- [11] S. Ji, J. Luo, and X. Yang, “A comprehensive survey on deep music generation: Multi-level representations, algorithms, evaluations, and future directions,” *arXiv preprint arXiv:2011.0680*, 2020.

- [12] C. Hernandez-Olivan and J. R. Beltran, “Music composition with deep learning: A review,” *Advances in speech and music technology: computational aspects and applications*, pp. 25–50, 2022.
- [13] A. Solanki and S. Pandey, “Music instrument recognition using deep convolutional neural networks,” *International Journal of Information Technology*, vol. 14, no. 3, pp. 1659–1668, 2022.
- [14] K. Racharla, V. Kumar, C. B. Jayant, A. Khairkar, and P. Harish, “Predominant musical instrument classification based on spectral features,” in *2020 7th International Conference on Signal Processing and Integrated Networks (SPIN)*, IEEE, 2020, pp. 617–622.
- [15] E. Manilow, G. Wichern, and J. Le Roux, “Hierarchical musical instrument separation,” in *ISMIR*, 2020, pp. 376–383.
- [16] P. Mangla, “Spotify music recommendation systems,” in *PyImageSearch*, P. Chugh, A. R. Gosthipaty, S. Huot, K. Kidriavsteva, and R. Raha, Eds., 2023.
- [17] Ableton. “Ableton live 11 lite.” (2024), [Online]. Available: <https://www.ableton.com/en/live/> (visited on 06/04/2024).
- [18] Apple. “Logic pro.” (2024), [Online]. Available: <https://www.apple.com/tw/logic-pro/> (visited on 06/04/2024).
- [19] PreSonus. “Studio one.” (2024), [Online]. Available: <https://www.presonus.com/en/studio-one.html> (visited on 06/04/2024).
- [20] Ronimusic. “Amazing slow downer.” (2024), [Online]. Available: <https://www.ronimusic.com/> (visited on 06/04/2024).
- [21] FORSCORE. “Forscore turbocharge tour sheet music.” (2024), [Online]. Available: <https://forscore.co/> (visited on 06/04/2024).
- [22] ISMIR. “International society for music information retrieval.” (2024), [Online]. Available: <https://ismir.net/> (visited on 05/06/2024).
- [23] X. Zhao, Q. Tuo, R. Guo, and T. Kong, “Research on music signal processing based on a blind source separation algorithm,” *Annals of Emerging Technologies in Computing (AETiC)*, vol. 6, no. 4, 2022.
- [24] Y. Mitsufuji, G. Fabbro, S. Uhlich, and F.-R. Stöter, *Music Demixing Challenge 2021*, 2021.
- [25] G. Fabbro, S. Uhlich, C.-H. Lai, *et al.*, “The Sound Demixing Challenge 2023 Music Demixing Track,” *arXiv e-prints*, arXiv:2308.06979, arXiv:2308.06979, Aug. 2023.
- [26] Z. Wang, K. Zhang, Y. Wang, *et al.*, “Songdriver: Real-time music accompaniment generation without logical latency nor exposure bias,” in *Proceedings of the 30th ACM International Conference on Multimedia*, 2022, pp. 1057–1067.
- [27] F. Ding and Y. Cui, “Museflow: Music accompaniment generation based on flow,” *Applied Intelligence*, vol. 53, no. 20, pp. 23 029–23 038, 2023.

- [28] C. Brazier and G. Widmer, “Improving real-time score following in opera by combining music with lyrics tracking,” *arXiv preprint arXiv:2110.02592*, 2021.
- [29] Antescofo. “Metronautapp.” (2024), [Online]. Available: <https://metronautapp.com/zh-TW> (visited on 06/04/2024).
- [30] P. Comon, “Independent component analysis, a new concept?” *Signal processing*, vol. 36, no. 3, pp. 287–314, 1994.
- [31] D. Lee and H. S. Seung, “Algorithms for non-negative matrix factorization,” *Advances in neural information processing systems*, vol. 13, 2000.
- [32] A. Maćkiewicz and W. Ratajczak, “Principal components analysis (pca),” *Computers & Geosciences*, vol. 19, no. 3, pp. 303–342, 1993.
- [33] F.-R. Stöter, S. Uhlich, A. Liutkus, and Y. Mitsufuji, “Open-unmix - a reference implementation for music source separation,” *Journal of Open Source Software*, vol. 4, no. 41, p. 1667, 2019.
- [34] Z. Rafii, A. Liutkus, F.-R. Stöter, S. I. Mimilakis, and R. Bittner, *The MUSDB18 corpus for music separation*, Dec. 2017.
- [35] Y. Luo and J. Yu, “Music Source Separation with Band-split RNN,” *arXiv e-prints*, arXiv:2209.15174, arXiv:2209.15174, Sep. 2022.
- [36] D. Stoller, S. Ewert, and S. Dixon, “Wave-u-net: A multi-scale neural network for end-to-end audio source separation,” *arXiv preprint arXiv:1806.03185*, 2018.
- [37] A. Défossez, “Hybrid spectrogram and waveform source separation,” *arXiv preprint arXiv:2111.03600*, 2021.
- [38] S. Rouard, F. Massa, and A. Défossez, “Hybrid transformers for music source separation,” in *ICASSP 2023-2023 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, IEEE, 2023, pp. 1–5.
- [39] M. Miron, J. Janer Mestres, and E. Gómez Gutiérrez, “Generating data to train convolutional neural networks for classical music source separation,” in *Lokki T, Pätynen J, Välimäki V, editors. Proceedings of the 14th Sound and Music Computing Conference; 2017 Jul 5-8; Espoo, Finland. Aalto: Aalto University; 2017. p. 227-33.*, Aalto University, 2017.
- [40] C.-Y. Chiu, W.-Y. Hsiao, Y.-C. Yeh, Y.-H. Yang, and A. Wen-Yu Su, “Mixing-Specific Data Augmentation Techniques for Improved Blind Violin/Piano Source Separation,” *arXiv e-prints*, arXiv:2008.02480, arXiv:2008.02480, Aug. 2020.
- [41] R. Hennequin, A. Khlif, F. Voituret, and M. Moussallam, “Spleeter: A fast and efficient music source separation tool with pre-trained models,” *Journal of Open Source Software*, vol. 5, no. 50, p. 2154, 2020.

- [42] M. Heydari and Z. Duan, “Don’ t look back: An online beat tracking method using rnn and enhanced particle filtering,” in *ICASSP 2021-2021 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, IEEE, 2021, pp. 236–240.
- [43] B. Di Giorgi, M. Mauch, and M. Levy, “Downbeat tracking with tempo-invariant convolutional neural networks,” *arXiv preprint arXiv:2102.02282*, 2021.
- [44] F. Henkel, S. Balke, M. Dorfer, and G. Widmer, “Score following as a multi-modal reinforcement learning problem.,” *Trans. Int. Soc. Music. Inf. Retr.*, vol. 2, no. 1, pp. 67–81, 2019.
- [45] P. Cano, A. Loscos, and J. Bonada, “Score-performance matching using hmms,” in *ICMC*, Citeseer, 1999.
- [46] H. Sakoe and S. Chiba, “Dynamic programming algorithm optimization for spoken word recognition,” *IEEE Transactions on Acoustics, Speech, and Signal Processing*, vol. 26, no. 1, pp. 43–49, 1978.
- [47] A. Arzt, G. Widmer, and S. Dixon, “Adaptive distance normalization for real-time music tracking,” in *2012 Proceedings of the 20th European Signal Processing Conference (EUSIPCO)*, 2012, pp. 2689–2693.
- [48] N. Takahashi, T. Yoshihisa, Y. Sakurai, and M. Kanazawa, “A parallelized data stream processing system using dynamic time warping distance,” in *2009 International Conference on Complex, Intelligent and Software Intensive Systems*, IEEE, 2009, pp. 1100–1105.
- [49] I.-C. Wei and L. Su, “Online music performance tracking using parallel dynamic time warping,” in *2018 IEEE 20th International Workshop on Multimedia Signal Processing (MMSP)*, 2018, pp. 1–6.
- [50] S. Dixon, “Live tracking of musical performances using on-line time warping,” in *Proceedings of the 8th International Conference on Digital Audio Effects*, Citeseer, vol. 92, 2005, p. 97.
- [51] A. Arzt and G. Widmer, “Towards effective ‘any-time’ music tracking,” in *Proceedings of the 2010 Conference on STAIRS 2010: Proceedings of the Fifth Starting AI Researchers’ Symposium*, NLD: IOS Press, 2010, pp. 24–36.
- [52] Y.-J. Lin, H.-K. Kao, Y.-C. Tseng, M. Tsai, and L. Su, “A human-computer duet system for music performance,” in *Proceedings of the 28th ACM International Conference on Multimedia*, ser. MM ’ 20, ACM, Oct. 2020.
- [53] Python. “Multiprocessing —process-based parallelism.” (2024), [Online]. Available: <https://docs.python.org/3/library/multiprocessing.html> (visited on 06/02/2024).
- [54] Python. “Pyaudio package.” (2024), [Online]. Available: <https://people.csail.mit.edu/hubert/pyaudio/> (visited on 06/02/2024).
- [55] B. McFee, M. McVicar, D. Faronbi, *et al.*, *Librosa/librosa: 0.10.2.post1*, 2024.

- [56] Z. K. Abdul and A. K. Al-Talabani, “Mel frequency cepstral coefficient and its applications: A review,” *IEEE Access*, vol. 10, pp. 122 136–122 158, 2022.
- [57] J. Thickstun, Z. Harchaoui, and S. M. Kakade, “Learning features of music from scratch,” in *International Conference on Learning Representations (ICLR)*, 2017.
- [58] J. Thickstun, Z. Harchaoui, D. P. Foster, and S. M. Kakade, “Invariances and data augmentation for supervised music transcription,” in *International Conference on Acoustics, Speech, and Signal Processing (ICASSP)*, 2018.
- [59] F. J. Muneratti Ortega, *Expressive solo violin*, 2021.
- [60] H.-W. Dong, C. Zhou, T. Berg-Kirkpatrick, and J. McAuley, *Bach violin dataset*, 2021.
- [61] R. Bittner, J. Salamon, M. Tierney, M. Mauch, C. Cannam, and J. Bello, “Medleydb: A multitrack dataset for annotation-intensive mir research,” Oct. 2014.
- [62] L. Yu-Jie. “音源分離資料集.” (2024), [Online]. Available: <https://drive.google.com/drive/folders/1IPGv2l-6QjIwMtAq9m0ijQ-ilvTCFjfU?usp=sharing> (visited on 06/24/2024).
- [63] E. Vincent, R. Gribonval, and C. Févotte, “Performance measurement in blind audio source separation,” *Audio, Speech, and Language Processing, IEEE Transactions on*, vol. 14, pp. 1462–1469, Aug. 2006.
- [64] L. Yu-Jie. “不同旋律的音源分離結果音檔.” (2024), [Online]. Available: <https://github.com/a0950088/Master/tree/main/paper/resource/4.1> (visited on 07/15/2024).
- [65] L. Yu-Jie. “系統在不同速度下的追蹤結果所使用的 midi 檔案與結果音檔.” (2024), [Online]. Available: <https://github.com/a0950088/Master/tree/main/paper/resource/4.2.1> (visited on 06/11/2024).
- [66] kopikostar. “Beethoven’s ”spring sonata” op.24 allegro.” (2024), [Online]. Available: <https://youtu.be/uDSfjK1qxo?list=PLc0i4xi7nsQRG0UTdRKRc2bxjh9pHYYJw> (visited on 06/03/2024).
- [67] L. Yu-Jie. “Beethoven 音源分離結果音檔.” (2024), [Online]. Available: <https://github.com/a0950088/Master/tree/main/paper/resource/4.2.2/music%20source%20separation> (visited on 06/11/2024).
- [68] L. Yu-Jie. “不同特徵下的追蹤結果音檔.” (2024), [Online]. Available: <https://github.com/a0950088/Master/tree/main/paper/resource/4.2.2> (visited on 06/11/2024).
- [69] 林承勳. “中央研究院研之有物: 今晚，想來場臨時音樂會？讓 ai 虛擬音樂家幫你實現！” (2021), [Online]. Available: <https://research.sinica.edu.tw/ai-virtual-musician-li-su/> (visited on 06/23/2024).